1. Coordinates, pinhole and calibration

Conventions throughout: column vectors; image arrays $I[y,x]$, x right/y down unless stated; world-to-camera $X_c = RX_w + t$; virtual image plane in front; visible $Z_c > 0$. A question's convention takes priority. $\sim$ means equal up to nonzero scale; $\odot$ is elementwise multiplication.

Pinhole: camera-frame origin = optical centre/pinhole; optical axis = z-axis. Similar triangles give $x = fX_c/Z_c$, $y = fY_c/Z_c$. Physical plane at $-f$ gives both minus signs (inverted image). Apparent fronto-parallel size $h = fH/Z$: twice f doubles size, twice depth halves it. A pixel gives a ray, not depth. For sensor width W, FOV $= 2\arctan(W/(2f))$; match units. Greater f means narrower FOV.

Optics: larger pinhole admits more light but increases geometric blur; tiny holes lose light and cause diffraction. A lens focuses rays from a chosen depth. Depth of field = acceptably sharp depth range; smaller aperture increases it, larger aperture decreases it (other settings fixed). Focus and focal length are different quantities.

Homogeneous coordinates: $(x,y) \mapsto (x,y,1)$; $(a,b,w) \mapsto (a/w, b/w)$ if $w \neq 0$. $w = 0$ is a point at infinity/direction, not a finite location. Common nonzero scale does not change a point. Translation becomes matrix multiplication; composition is simple, rightmost first.

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad P = K[R \mid t].$$

$f_x = f/p_x$, $f_y = f/p_y$ for pixel pitches p_x, p_y; use matching physical units. (c_x, c_y) = principal point (optical-axis intersection, not necessarily image centre). Skew s models nonorthogonal sensor axes, usually 0. K: 3×3, 5 DoF; R: 3×3, 3 DoF; t: 3×1, 3 DoF; extrinsic block: 3×4; P: 3×4, 11 DoF up to scale.

$$\tilde{x} \sim P \begin{bmatrix} X_w \\ 1 \end{bmatrix}, \quad u = \frac{f_x X_c + s Y_c}{Z_c} + c_x, \quad v = \frac{f_y Y_c}{Z_c} + c_y.$$

Always compute camera coordinates before perspective division. Neither the 4D world point's last entry nor focal length is the denominator: use projected third entry (Z_c for this K).

Pose/centre: $R^\top R = I$, $\det R = +1$, $R^{-1} = R^\top$. Camera centre $C = -R^\top t$, so $t = -RC$; t is *not* generally camera position. Camera-to-world is $X_w = R^\top X_c + C$. If camera-to-world orientation is Q, world-to-camera R is $Q^\top$.

$$T_{cw} = \begin{bmatrix} R & t \\ 0\,0\,0 & 1 \end{bmatrix}, \quad T_{cw}^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0\,0\,0 & 1 \end{bmatrix}.$$

Points use R and t; directions/normals use only R: $n_c = Rn_w$. Solve $Kd = \tilde{x}$ for a camera ray; world ray $C + \lambda R^\top K^{-1}\tilde{x}$, $\lambda > 0$. Normal transformation under a general nonrigid A uses $A^{-\top}$, then normalize.

Right-handed active rotations, c=cosθ, s=sinθ:

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix}, \quad R_y = \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix}.$$

$$R_z = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Camera rotation vs point rotation can require a transpose. y-down changes apparent rotation direction. Rotate then translate: TR; translate then rotate: RT (rotate the translation too). In 3D, $R_1R_2 \neq R_2R_1$ in general.

Basic trigonometric values (angles in degrees):

θ	0°	30°	45°	60°	90°
$\sin\theta$	0	1/2	$\sqrt{2}/2$	$\sqrt{3}/2$	1
$\cos\theta$	1	$\sqrt{3}/2$	$\sqrt{2}/2$	1/2	0

$\sqrt{2}/2 \simeq .7071$, $\sqrt{3}/2 \simeq .8660$. NumPy uses radians: $\theta_{rad} = \theta_{deg}\pi/180$, e.g. `np.sin(np.deg2rad(30))=.5`.

Crop/resize: crop top-left (x_0, y_0), then resize (a_x, a_y):

$$K' = \begin{bmatrix} a_x & 0 & -a_x x_0 \\ 0 & a_y & -a_y y_0 \\ 0 & 0 & 1 \end{bmatrix} K.$$

Thus $f'_x = a_x f_x, s' = a_x s, c'_x = a_x(c_x - x_0)$, similarly y. Extrinsics do not change. This ignores half-pixel resampling offsets; follow a stated pixel-centre rule.

Orthographic: parallel rays, $(X,Y,Z) \mapsto (X,Y)$ (possibly scaled/-translated), no Z-dependent size; suitable distant-view approximation. Perspective causes foreshortening; parallel 3D lines can meet at a vanishing point. Plane directions' vanishing points form a vanishing line/horizon; $v \sim KRd_w$.

Distortion/calibration: radial distortion gives barrel/pincushion effects; tangential distortion comes from misalignment. These are outside ideal K. Undistort before fitting ideal lines/H/F. Calibration estimates intrinsics, distortion and poses from known geometry plus observations; more data cannot repair a degenerate configuration.

2. Sensor, sampling and colour formation

Representation: grayscale $I : \mathbb{Z}^2 \to \mathbb{R}$; RGB $\mathbb{Z}^2 \to \mathbb{R}^3$; binary $\{0,1\}$; labels = discrete class IDs. Spatial sampling discretizes position; quantization discretizes values. b bits give 2^b levels $0, \ldots, 2^b - 1$; uint8: 256 levels, 0–255. Float is not automatically [0,1]. Sensor noise and saturation limit dynamic range; rolling shutter exposes rows at different times and can distort motion.

Sensor colour: $c_k \propto \int E(\lambda)\rho(\lambda)S_k(\lambda)\,d\lambda$, plus orientation, visibility, gain/response. E: source spectrum; ρ: material reflectance; S: sensor sensitivity. Different spectra can produce equal RGB (metamers); RGB does not uniquely recover spectra. CCD = Charge-Coupled Device (photons to accumulated charge); retina rods/cones measure light. OpenCV normally loads BGR; display/coefficients may require RGB.

Vectorized projection of N world points stored as rows:

```
Xc = Xw @ R.T + t.reshape(1,3)
qh = Xc @ K.T
uv = qh[:, :2] / qh[:, 2:3] # mask Zc<=0
C = -R.T @ t
ray_c = np.linalg.solve(K, [u,v,1.])
ray_w = R.T @ ray_c
```

Row-vector storage changes multiplication placement, not pose convention. `Xw @ R` would use the wrong rotation here. For camera at $(2,0,0)$, R=I gives $t = (-2,0,0)$; point $(5,1,10)$ becomes $(3,1,10)$. With $f_x = f_y = 100$, $c_x = c_y = 0$, pixel = (30,10), not (70,10). Moving camera +x shifts stationary scene -x.

Units: f=5 mm and pitch=10 micrometres=0.01 mm gives $f_x = 500$ pixels. Resizing cannot change physical focal length, but does change focal length in pixel units. Changing the world origin changes extrinsics, not intrinsic K.

3. Colour spaces, grayscale and constancy

Grayscale: average $(R+G+B)/3$; lightness $(\max+\min)/2$; usual luma approximation $0.299R + 0.587G + 0.114B$. Use supplied coefficients, not a universal physical-luminance claim. For BGR reverse coefficient order. Convert uint8 to float before sums/differences; unsigned subtraction can wrap (e.g. $0-1 \to 255$).

Normalized RGB: $r = R/(R+G+B)$, similarly g,b; $r+g+b = 1$, 2 independent values. Removes common positive multiplicative gain, not channel-dependent gain, offsets, clipping or arbitrary lighting. Black pixels need a mask/fallback. Averaging normalized colours $\neq$ normalizing their average.

Opponent: $O_1 = (R-G)/\sqrt{2}$, $O_2 = (R+G-2B)/\sqrt{6}$, $O_3 = (R+G+B)/\sqrt{3}$: red–green, yellow–blue, intensity. Differences may be negative; display scaling is separate. YCbCr separates luma Y and blue-/red-difference chroma Cb/Cr; exact coefficients/offsets depend on standard/range.

HSV: hue cyclic colour angle; saturation distance from gray; value $V = \max(R,G,B)$, not HSL lightness. Let $M = \max$, $m = \min$, $\Delta = M - m$; $S = \Delta/M$ if $M > 0$, else 0. Hue is undefined when $\Delta = 0$. With normalized RGB, hue in degrees:

$$H = 60\times\begin{cases} ((G-B)/\Delta) \bmod 6 & M = R \\ (B-R)/\Delta + 2 & M = G \\ (R-G)/\Delta + 4 & M = B. \end{cases}$$

Lab/OpenCV uint8 HSV: H=0–179 (half-degrees), S,V=0–255; do not mix with float [0,1] or ordinary degrees. Separate brightness/chroma operations can reduce colour changes, not guarantee illumination invariance.

Colour constancy: estimate and remove illuminant cast. Grey-World assumes neutral scene-average reflectance; spatial means m_c, target $\bar{m} = (m_R + m_G + m_B)/3$, then $I'_c = I_c\bar{m}/m_c$. Monochromatic scenes, zero means, clipping/nonuniform light cause failures. White-Patch (optional example for Lab1's open choice) assumes a brightest/reference patch is neutral; vulnerable to coloured highlights/saturation.

```
I = I.astype(float)
mc = I.mean(axis=(0, 1), keepdims=True)
out = I / mc * mc.mean()  # mask mc==0
out = np.clip(out, 0, 255).astype(np.uint8)
gray = (I.max(-1) + I.min(-1)) / 2
rgb = bgr[..., ::-1].copy()
```

$(0,1)$ reduces H,W and retains channels. Reducing $(0,1,2)$ gives the same scalar as target and cancels correction. mean(axis = 2) instead gives one grayscale value per pixel.

4. Reflection and the BRDF

Normal n: unit surface-perpendicular vector. **Albedo** ρ: material diffuse reflectance, calibrated physical $0 \le \rho \le 1$. **Irradiance**: incoming power/area. **Radiance**: directional power/(projected area×solid angle), related to camera signal. Distant directional light has constant parallel direction; strength may be vector magnitude.

Lambertian: with unit vectors toward light ℓ and viewer v,

$$I = \rho J \max(0, n^\top \ell).$$

Cosine law: maximum face-on; zero at 90°, clamp back-facing values. Radiance is view-independent, but visibility/projected area is not. Light strength J and camera gain can be absorbed in L/effective ρ. Physical Lambertian BRDF = ρ/π; do not insert π into an already absorbed course coefficient.

Diffuse vs specular: body scattering yields matte diffuse appearance; surface reflection yields view-dependent glossy/mirror highlights. Phong sums ambient, diffuse and specular:

$$I = k_a I_a + J[k_d \max(0, n^\top \ell) + k_s \max(0, v^\top r)^s].$$

$r = 2(n^\top \ell)n - \ell$; higher shininess s gives narrower highlights. At mirror-sphere highlight, $\ell = 2(n^\top v)n - v$; known sphere geometry gives n. Normalize directions and state outward-vector signs.

Seminar colour invariants: under white light, diffuse $(R,G,B) = a(k_R, k_G, k_B)$; colours lie on a ray through RGB origin, $R/G = k_R/k_G$. Equal-channel specular term b gives $(R,G,B) = a\mathbf{k} + b\mathbf{1}$; $(R-G)/(R-B) = (k_R - k_G)/(k_R - k_B)$. Valid nonzero denominators, linear response and shared scalar factors required; coloured specular light/clipping can break invariance.

BRDF = Bidirectional Reflectance Distribution Function, $f_r(\ell, v) = dL_o(v)/dE_i(\ell)$: incoming and outgoing directions each need two angles (4D angular function). Reciprocal materials satisfy $f_r(\ell, v) = f_r(v, \ell)$ (Helmholtz reciprocity). BRDF angular arguments depend on local normal: general reflectance inversion is nonlinear.

$$L_o(v) = \int_{\Omega^+} f_r(\ell, v) L_i(\ell)(n^\top \ell)\,d\omega.$$

The simple directional-light model is a special case; shadows, interreflection, transparency and nonlinear camera response are not explained by one dot product.

Colour/lighting calculation recipes

```
S = I.sum(axis=-1, keepdims=True) # float RGB
rgbn = np.divide(I, S, out=np.zeros_like(I),
                 where=S>0)
o1 = (I[...,0] - I[...,1]) / np.sqrt(2)
o2 = (I[...,0]+I[...,1]-2*I[...,2])/np.sqrt(6)
# Unit normal from surface tangent vectors a,b
n = np.cross(a, b)
n = n / np.linalg.norm(n) # nonparallel a,b
Id = rho * J * max(0., np.dot(n, light))
```

Grey-World means (4,2,2) give target 8/3 and gains (2/3,4/3,4/3), corrected means equal 8/3 before clipping. Dark/zero channels cannot be corrected by division. Material $\rho = .4$, light J=200, angle 60° gives diffuse intensity 40; face-on gives 80. Albedo alone is not measured brightness.

Light models: distant directional strength constant; point-light irradiance additionally decreases with squared distance and direction changes across object. A calibrated single L row per image assumes its direction/strength applies across the modeled surface; nearby-light effects need a different model. Lambertian is view-independent, not independent of light or normal.

5. Intrinsic images and shape from shading

Intrinsic image model: $I = A \odot S$, material/albedo A times shading S; possible extra specular layer $I = A \odot S + H$. Recolour A inside object mask, leave S fixed, multiply (not divide) to reconstruct. Match [0,1] vs [0,255] units: normalized multiplicative layers avoid an unwanted extra factor 255. Synthetic rendering can expose exact layers; synthetic performance is not proof on real scenes.

```
mask = np.any(albedo != 0, axis=-1)
A = albedo.astype(float).copy()
A[mask] = [1., 0., 0.]  # normalized RGB
recolored = A * shading # compatible shapes
```

Shape from shading: single-view intensity mixes light, material and orientation. Usual assumptions: diffuse constant albedo, known distant light, orthographic camera; still needs boundary/prior/integrability constraints. One intensity constrains n to a cone, not a unique direction. Texture/material changes must not be mistaken for shape.

6. Calibrated photometric stereo (Lab1)

Setup: fixed object and camera, multiple lighting directions. Unlike geometric stereo (different viewpoints), it recovers local normals/albedo first, then height by integration. For m lights stacked as rows $L \in \mathbb{R}^{m\times 3}$, observations $i \in \mathbb{R}^m$, $g = \rho n$:

$$i = Lg, \quad \hat{g} = \arg\min_g \|Lg - i\|_2^2 = L^+ i.$$

Full column rank: $\hat{g} = (L^\top L)^{-1} L^\top i$; use stable least squares, not explicit inversion. At least 3 linearly independent valid lights; rank $L = 3$. Three images alone are insufficient. Repeating dependent lights cannot reveal nullspace components. More independent measurements can reduce noise and reveal residual errors, not remove systematic model error.

Recover: $\rho = \|\hat{g}\|_2$, $n = \hat{g}/\rho$ for $\rho > 0$. Unknown common positive light/gain factor scales ρ, not n. Colour-channel fits should agree on geometry; a zero-reflectance channel gives $g = 0$, no valid direction. Light columns/pixels-as-rows are also possible: follow shapes before transposing.

```
g = np.linalg.lstsq(L, i, rcond=None)[0]
rho = np.linalg.norm(g)
n = g / rho                 # only if rho > 0
# All N pixels: obs is (m,N)
G = np.linalg.lstsq(L, obs, rcond=None)[0]
albedo = np.linalg.norm(G, axis=0)
normals = G / albedo[None, :]  # mask zeros
```

Residuals: $r = Lg - i$; least squares minimizes squared error, not necessarily exact equality. Overdetermined inconsistent data leaves residuals. Large residuals suggest bad lights, shadows, specularity, motion, interreflection or nonlinearity; small residuals do not prove physically correct geometry.

Weighted LS: $\min \sum_j \alpha_j (L_j g - i_j)^2$; multiply each equation by $\sqrt{\alpha_j}$. If using row factors w_j, actual weights are w_j^2.

$$g = (L^\top W L)^{-1} L^\top W i, \quad W = \mathrm{diag}(\alpha_j).$$

Lab1 shadow trick multiplies by intensity: $\mathrm{diag}(i)Lg = \mathrm{diag}(i)i$. Thus $W = \mathrm{diag}(i_j^2)$: dark equations count less; bright specular outliers can count *more*. Weighting is a heuristic, not automatic robust recovery; retain rank after removing/downweighting observations.

```
gw = np.linalg.lstsq(i[:,None]*L, i*i,
                     rcond=None)[0]
```

7. Normals to height: signs and integrability

For upward height graph $z(x,y)$, p=z_x, q=z_y:

$$n = \frac{(-p,-q,1)^\top}{\sqrt{1+p^2+q^2}}, \quad p = -n_x/n_z, \quad q = -n_y/n_z.$$

Source conflict: HC2a uses these minus signs; Lab1 prints positive ratios. For Lab1 code interpretation use its stated definition; for geometry use declared axes/normal convention. Reversing the whole normal does not change n_x/n_z. Near $n_z = 0$ slopes are unstable/vertical, and a single-valued height graph may fail.

Integrate: $dz = p\,dx + q\,dy$; discrete $z[y,x+1] - z[y,x] \simeq p[y,x]\Delta x$, $z[y+1,x] - z[y,x] \simeq q[y,x]\Delta y$. Start from an anchor and accumulate along a stated path. With pixel steps 1, sums give relative heights; with physical spacing multiply slopes by spacing.

Integrability: $p_y = q_x$ (smooth field, simply connected domain) gives path independence. Noisy/non-Lambertian normals may violate it; different row-first/column-first paths can disagree and accumulate drift. Valid gradients determine z only up to an additive constant.

Global integration: solve all neighbouring height-difference equations in LS with one anchor. Equivalent continuous normal equation $\nabla^2 z = p_x + q_y$ with boundary/gauge constraints. Distributes inconsistency rather than committing to one path; does not create missing absolute height.

Sanity checks: $\|n\| = 1$; L's column rank; correct observation/light row order; valid object mask; consistent light-direction signs; input linearization; calibrated light strengths. Flat front-facing surface $n = (0,0,1)$ gives $p = q = 0$; $z = ax + by + c$ gives $n \propto (-a,-b,1)$.

One integration path (unit spacing, left/top differences assigned to outgoing edges, anchor z[0,0]=0):

```
z = np.zeros_like(p, dtype=float)
z[0,1:] = np.cumsum(p[0,:-1])
for y in range(1, z.shape[0]):
    z[y,:] = z[y-1,:] + q[y-1,:]
# Opposite path may differ if p_y != q_x
qy, qx = np.gradient(q) # axis 0=y, axis 1=x
py, px = np.gradient(p)
curl_error = py - qx
```

An upward normal $(-2,1,2)/3$ gives p=1,q=-1/2; moving 4 x-units and 2 y-units changes height by $4 - 1 = 3$. Multiplying n by -1 leaves both ratios unchanged. Nonintegrability is about mixed derivatives, not whether p and q individually vary.

Weight shapes: `sqrt(alpha)[:,None]*L` scales light rows; `sqrt(alpha)*i` scales matching observations. Multiply *both* sides. Intensity-derived weights differ per pixel; one common weighted inverse generally cannot serve all pixels.

1. Histograms, point operations and thresholding

Histogram: $h(r) = \#\{I = r\}$; normalized $p(r) = h(r)/N$, $\sum p = 1$; CDF $F(r) = \sum_{j \le r} p(j)$, nondecreasing, ends at 1. Counts reveal intensity distribution, not layout, edges or connectivity; multiple images share h. Too few bins merge distinctions, too many give sparse/noisy estimates. Separate RGB histograms lose cross-channel association; joint histograms preserve association, still not spatial layout.

Point operator: $s = T(r)$, output depends only on input at same pixel (mapping may depend on global statistics). Linear contrast stretch $s = c + (r-a)(d-c)/(b-a)$, $b > a$; clip if specified. $s = \alpha r + \beta$: α contrast, β brightness. Negative for levels $0 \dots L-1$: $s = L-1-r$. Gamma $s = cr^{\gamma}$, normalized $0 < r < 1$: $\gamma < 1$ brightens, $\gamma > 1$ darkens. Some systems define inverse gamma; read the formula.

Histogram equalization: hand procedure (S1 Q5c–f). 1. Count: for b-bit image, $L = 2^b$; list every level $k = 0, \dots, L-1$, including zero-count bins. $h_k = \#\{I = k\}$, $N = \sum_k h_k$ (or H×W). **2. Normalize:** $p_k = h_k/N$. **3. Cumulate:** $F_k = \sum_{j=0}^{k} p_j$; inclusive running sum, last value1. **4. Map:** multiply by $L-1$, not L or255 for a 3-bit image. Seminar uses nearest integer, *no CDF-min subtraction*:
$T(k) = \text{round}[(L-1)F_k]$.
5. Merge counts: $h'_s = \sum_{k:T(k)=s} h_k$; add all old counts mapping to s, don't overwrite them. Draw output histogram with x=0…$L-1$, y=h'_s; $\sum h' = N$. **6. Back-project:** replace each original pixel k by T(k): $J[y,x] = T(I[y,x])$. Positions stay fixed; histogram alone cannot recover them. Compute exact cumulative counts before final rounding.

Other convention, only if stated:
$$s(r) = \text{round}\left[\frac{(L-1)(F(r) - F_{\min})}{1 - F_{\min}}\right].$$
$F_{\min}$=first positive CDF; constant image needs a fallback. Floor versus round changes output; `np.rint` uses ties-to-even. Identical input values cannot split, so a discrete equalized histogram need not be flat; global enhancement can amplify noise/neglect local contrast.

```
h = np.bincount(I.ravel(), minlength=L)
cdf = h.cumsum() / h.sum()
lut = np.rint((L-1)*cdf).astype(int)
J = lut[I]  # I must contain integer levels
h_new = np.bincount(lut, weights=h, minlength=L)
```

S1 Q5f RGB: RGB→HSV; equalize only V; keep H,S; recombine H,S,V' and convert back to RGB. Retains HSV hue/saturation under ideal conversion (rounding/gamut can affect results). Independent R/G/B equalization changes channel ratios and can change hue. **Threshold:** $B[y,x] = 1$ if $I[y,x] \ge T$ (or specified strict rule); binarization discards graded appearance. Lighting/shadows can alter B; component count is not automatically object count.

2. Connected components and morphology

Pixel graph: vertices = foreground pixels; edges = allowed adjacency. 4-neighbours: up/down/left/right. 8-neighbours also diagonals; diagonal contact joins only under 8-connectivity. Component = maximal foreground-connected set. Touching objects can merge; one broken object can split. Foreground/background connectivity must be specified; using complementary 4/8 rules can avoid digital-topology ambiguities.

Two-pass labelling: raster scan; 4-connectivity checks already-seen left/up; 8 checks left, upper-left, up, upper-right. No labelled neighbour: new provisional label. Otherwise use one neighbour label and record equivalences among all touching labels. Second pass resolves equivalence classes (e.g. union–find) and assigns one label per component. Flood-fill/BFS/DFS is an alternative. Count labels excluding background.

Structuring element K: active offsets with an origin. Zeros are ignored, *not* requirements that corresponding image pixels are 0. For binary set B, $K_z = \{z + k : k \in K\}$:
$B \oplus K = \{b + k : b \in B, k \in K\}$, $B \ominus K = \{z : K_z \subseteq B\}$.
Thus dilation at z tests $B(z-k)$ (any active k); erosion tests $B(z+k)$ (all k). Symmetric K makes reflected/nonreflected versions equal; for asymmetric K use the stated convention. Grayscale flat dilation = max; erosion = min over correct offsets.

Dilation: grows foreground, joins regions, fills small gaps/holes. **Erosion:** shrinks/removes small foreground structures, breaks thin bridges. With origin in K, dilation is extensive and erosion anti-extensive (consistent whole-domain definitions).
$B \circ K = (B \ominus K) \oplus K$ (opening),
$B \bullet K = (B \oplus K) \ominus K$ (closing).
Opening removes small foreground/protrusions/bridges; closing fills small background gaps/cracks and may join regions. Erosion/dilation are not inverses. Ideal opening/closing are idempotent, $(B \circ K) \circ K = B \circ K$, and open$\subseteq B \subseteq$close. Cropped finite intermediates/padding can change these ideal properties.

Boundary: zero, mirror, clamp and wrap differ; apply at *each* stage. Zero-padded 3×3 erosion of all ones removes border pixels. Closing can lose border foreground because dilation beyond cropped frame is discarded. Larger K means stronger size-dependent effects; cross vs square K changes diagonal coverage. Complement duality: $(B \oplus K)^c = B^c \ominus \hat{K}$, with reflected $\hat{K} = -K$.

Measurements: area $A = \sum B$; centroid $(\bar{x}, \bar{y}) = (\sum xB/A, \sum yB/A)$; physical area = pixel count×pixel area. Inner boundary $B - (B \ominus K)$; outer $(B \oplus K) - B$; morphological gradient $(B \oplus K) - (B \ominus K)$. Threshold → opening/separation → labelling can count objects, but may destroy genuine small ones.

Discrete calculations and morphology code

S1 Q5c worked table: L=8, N=4400, nearest integer:

k	0	1	2	3	4	5	6	7
h_k	900	800	700	600	500	400	300	200
$7F_k$	1.43	2.70	3.82	4.77	5.57	6.20	6.68	7.00
$T(k)$	1	3	4	5	6	6	7	7

$h' = (0, 900, 0, 800, 700, 600, 900, 500)$: output6 merges 500+400; output7 merges 300+200. **Q5d–e patch:** count $h = (16, 19, 14, 7, 6, 2, 0, 0)$, N=64; $T = (2, 4, 5, 6, 7, 7, 7, 7)$, $h' = (0, 0, 16, 0, 19, 14, 7, 8)$. First row $(1, 1, 1, 3, 3, 4, 4, 1) \to (4, 4, 4, 6, 6, 7, 7, 4)$; apply lookup to all other pixels likewise.

Symmetric element, zero-padding, odd K: reduce only active mask positions; full sliding windows below give one output/input pixel.

```
from numpy.lib.stride_tricks import sliding_window_view
r = K.shape[0]//2          # square, odd support
W = sliding_window_view(np.pad(B, r), K.shape)
dilated = np.any(W[..., K.astype(bool)], axis=-1)
eroded = np.all(W[..., K.astype(bool)], axis=-1)
# Opening: recompute windows after erosion
# Closing: recompute windows after dilation
```

For all-ones 5×5 B and 3×3 all-active K: zero-padded erosion keeps 9 pixels; dilation within frame keeps 25. Opening restores 25, while cropped zero-padded closing leaves 9–border handling can defeat ideal extensivity. In 3×3 identity foreground, 4-connectivity gives 3 components, 8 gives 1; area=3, centroid=(1,1) in zero-based x,y.

3. Spatial filtering and noise (Lab2)

Linear: $H(aI + bJ) = aH(I) + bH(J)$. Fixed unbiased weighted sums are linear; fixed nonzero bias makes an affine map. Median/max/thresholding are nonlinear. Shift invariance means identical translated neighbourhood rules (boundaries can break it).

$$(I * K)[y,x] = \sum_{a,b} K[a,b] I[y-a, x-b],$$

$$(I \star K)[y,x] = \sum_{a,b} K[a,b] I[y+a, x+b].$$

Convolution flips kernel in both axes; cross-correlation does not. CN-N/OpenCV routines often call correlation "convolution". For symmetric K they agree. For odd antisymmetric derivative K they differ by sign; gradient magnitude and unsigned HOG agree, signed angles differ by 180°.

Modes: valid requires full support, size $(H - K_h + 1) \times (W - K_w + 1)$; full $(H + K_h - 1) \times (W + K_w - 1)$; same matches input grid using stated padding/origin. Even kernels need alignment conventions. Zero padding can darken constant borders; reflect/replicate preserve constant values, wrap is periodic. Mirror variants can include/exclude edge pixels; inspect the rule.

Box/mean: K×K all $1/K^2$, low-pass; larger support smooths more but blurs edges. Smoothing kernel sum 1 preserves constants in valid regions. Sequential linear convolutions combine by kernel convolution: $(I * K_1) * K_2 = I * (K_1 * K_2)$; resulting support per axis $K_1 + K_2 - 1$. Order commutes ideally, not necessarily with intermediate cropping/padding.

Gaussian:
$$g_\sigma(x) = \frac{e^{-x^2/(2\sigma^2)}}{\sqrt{2\pi}\sigma}, \quad G_\sigma(x,y) = \frac{e^{-(x^2+y^2)/(2\sigma^2)}}{2\pi\sigma^2}.$$
Sample a finite centred grid (often radius $\lceil 3\sigma \rceil$), normalize by actual sum; sigma in pixel units unless stated. $G(x,y) = g(x)g(y)$ is separable. Anisotropic $G = g_{\sigma_x}(x) g_{\sigma_y}(y)$ still separable. Rank-1 kernel $K = ab^{\mathsf{T}}$ allows two 1D passes. Direct cost $O(HWK^2)$, separable $O(HWK)$: K^2 vs about 2K products/pixel.
$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2 + \sigma_2^2}}$.
Variances add, not sigmas. Extra blur to target scale: $\sigma_{add} = \sqrt{\sigma_{target}^2 - \sigma_{current}^2}$, only if target≥current. Larger sigma suppresses fine detail/noise and lowers effective frequency cutoff; finite sampled implementations approximate these identities.

Median: middle sorted neighbourhood value (specified even-sample convention), robust to sparse salt-and-pepper outliers, nonlinear, may remove thin structures. Mean/Gaussian can reduce random additive noise but spread impulse outliers and blur transitions. No filter is universally best; noise type/scale and signal detail matter.

4. Derivatives, gradients and scale

Gradient filters: central difference correlation $D_x = [-1, 0, 1]/2$, $D_y = D_x^{\mathsf{T}}$, so $G_x[y,x] \simeq (I[y, x+1] - I[y, x-1])/2$, $G_y[y,x] \simeq (I[y+1, x] - I[y-1, x])/2$. **S3 Q2a uses** $F_x = [1, 0, -1]$, $F_y = F_x^{\mathsf{T}}$, no factor1/2, correlated with the patch. Thus $G_x = I_{left} - I_{right}$, $G_y = I_{up} - I_{down}$. It uses the interior 3×3 centres of the 5×5 patch; don't introduce padded-border votes. Opposite derivative sign leaves magnitude/unsigned HOG unchanged, but rotates signed direction by180°.

Standard choices: Sobel smooths perpendicular to derivative; Prewitt uses equal perpendicular weights:
$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad P_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = S_x^{\mathsf{T}},\ P_y = P_x^{\mathsf{T}}.$$
Sobel separable $S_x = [1,2,1]^{\mathsf{T}}[-1,0,1]$; Prewitt $[1,1,1]^{\mathsf{T}}[-1,0,1]$. Gaussian derivatives $I * G_x$, $I * G_y$ smooth and differentiate. Keep kernel sign, scale, padding and correlation/convolution conventions consistent. **Gradient magnitude** = Euclidean length of the per-pixel derivative vector, measures intensity-change strength independent of its direction. First square each derivative, add *at that pixel*, then square-root (not sum across the image):
$$\nabla I = (I_x, I_y), \quad m = \sqrt{I_x^2 + I_y^2}, \quad \theta = \text{atan2}(I_y, I_x).$$
Compute `np.sqrt(Gx**2+Gy**2)` or `np.hypot(Gx,Gy)` in float. (3,4) gives m=5; (-3,-4) also5. m^2 is squared magnitude, not magnitude. Laplacian $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ is a *second*-derivative filter, not the gradient vector. The true/default-sign gradient points toward maximum intensity increase, normal to edge tangent; S3's opposite-sign filter reverses that vector. Large x derivative often indicates a vertical edge. Smooth before differentiating; derivative kernels sum0 to remove constants away from boundaries.

$$\partial_x (I * G) = I * (\partial_x G), \quad g'_\sigma = -\frac{x}{\sigma^2} g_\sigma,$$
$$g''_\sigma = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) g_\sigma, \quad \nabla^2 G_\sigma = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} G_\sigma.$$
First derivative extrema correspond to second derivative zero-crossings for ideal smoothed steps; require sufficient response, not every zero-crossing. Scale-normalized mth derivative multiplies by σ^m. LoG = Laplacian of Gaussian. **Difference** of Gaussians $G_{k\sigma} - G_\sigma \simeq (k-1)\sigma^2 \nabla^2 G_\sigma$ for k near 1; band-pass/blob-scale response. It is not a spatial **derivative** of Gaussian.

Gabor (taught definition): Gaussian envelope times sinusoidal carrier, oriented/local-frequency analysis: $e^{-(x'^2+\gamma^2 y'^2)/(2\sigma^2)} \cos(2\pi x'/\lambda + \phi)$, with sine imaginary partner; rotated coordinates x',y'. Advanced filter-bank implementation is optional.

Kernel construction and manual response recipes

```
x = np.arange(-r, r+1, dtype=float)
g = np.exp(-x*x/(2*sigma*sigma))
g /= g.sum()
G = np.outer(g, g)       # sum 1, separable
gx = -x/(sigma*sigma)*g # spatial derivative
Gx = np.outer(g, gx)     # rows=y, columns=x
P = np.pad(I.astype(float), r)
W = sliding_window_view(P, G.shape)
Jcorr = np.einsum('yxab,ab->yx', W, G)
Jconv = np.einsum('yxab,ab->yx', W, G[::-1,::-1])
```

`sliding_window_view` imported at left; r=kernel radius, sigma>0. gx uses sampled g; derivative sum≈ 0, do not normalize to sum1. Flip asymmetric kernels for convolution.

Seminar kernel W: $\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} [1, -2, 1]$: second difference in x and y, approximates mixed fourth derivative I_{xxyy}; responds to variation in both axes, not a general guaranteed corner/blob classifier. Combining it with 3×3 box uses a 5×5 kernel, not a 3×3 elementwise product.

Independent equal-variance noise: averaging n samples divides variance by n, standard deviation by $\sqrt{n}$; not true for correlated noise.

5. Fourier analysis: complex numbers to filtering

Meaning: weighted spatial sinusoids/complex exponentials; magnitude = amount, phase = spatial placement. Low frequency: slow broad variation; high: fine detail/edges (also noise). DC = constant component. Complex coefficients are needed even for real images: encode cosine and sine/phase jointly.
$e^{i\theta} = \cos\theta + i \sin\theta$, $z = a + ib = Ae^{i\phi}$,
$A = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$, $a = A\cos\phi$, $b = A\sin\phi$.
A^2 is power, not magnitude. Phase at A=0 has no physical meaning. $\arctan(b/a)$ loses quadrants/divides by zero. **NumPy order:** `atan2(imag,real)`; HC2b's printed argument order is reversed. $Ae^{i(\omega x + \phi)}$: radius A, angular rate ω, initial angle ϕ; frequency f, $\omega = 2\pi f$, period $T = 1/f$. Negative exponent rotates clockwise in complex plane.

DFT convention (NumPy default, indices 0-based):
$$F[k] = \sum_{n=0}^{N-1} f[n] e^{-2\pi i k n/N},$$
$$f[n] = \frac{1}{N} \sum_{k=0}^{N-1} F[k] e^{2\pi i k n/N}.$$
$$F[v,u] = \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} I[y,x] e^{-2\pi i (vy/H + ux/W)}.$$
2D inverse divides by HW. $F[0] = \sum f$, $F[0,0] = \sum I$; mean = DC/sample count. Frequency k means k cycles across N samples, k/N cycles/pixel, interpreted with modulo/negative frequencies. Real input: $F[-k] = \overline{F[k]}$ (mod N); in 2D $F[-v,-u] = \overline{F[v,u]}$. DC and even-length Nyquist coefficients are real. Other normalizations change coefficient scales.

Theorems: spatial convolution ↔ pointwise spectral multiplication; spatial multiplication ↔ frequency convolution with normalization factors. Spatial translation by Δ multiplies spectrum by $e^{-i\omega\Delta}$: magnitude unchanged, phase changed. Phase holds arrangement; magnitude alone cannot reconstruct a general image. Discarding imaginary coefficients loses phase. Linearity applies in both domains.

Boundary/padding: finite DFT multiplication yields *circular* convolution (periodic extension). For full linear convolution lengths N,M, pad to at least $N + M - 1$; 2D pad to $H + K_h - 1$, $W + K_w - 1$. Place kernel origin consistently (centred kernels need correct shift before FFT). Zero-padding does not add missing measurements.

Frequency filters: multiply F by transfer H then inverse transform. Low-pass keeps central low frequencies; high-pass removes DC/slow trends; band-pass selects a range. Ideal sharp cutoff can ring; Gaussian smoother cutoff reduces this. Derivatives multiply continuous spectra by $i\omega_x$ or $i\omega_y$; discrete differences have their own transfer function.
Parseval for this DFT: $\sum_n |f[n]|^2 = (1/N) \sum_k |F[k]|^2$.

Examples: low-pass Gaussian G_σ or box $\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$, keeps DC/slow structure, blurs detail. **High-pass** Laplacian $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, removes constants/emphasizes rapid changes. **Band-pass** Difference of Gaussians $G_{\sigma_1} - G_{\sigma_2}$, $\sigma_2 > \sigma_1$: cancels DC, passes intermediate scales, suppresses very fine noise. Gabor is another oriented frequency-selective example.

Sampling: Nyquist limit = half sampling rate; spatial frequencies above it alias into lower ones. Low-pass before decimation; factor a downsample reduces admissible original-grid frequency to $1/(2a)$ cycles/pixel. Resizing without appropriate antialiasing can create moiré. FFT is fast DFT evaluation, typically $O(N \log N)$, not a different transform.

```
F = np.fft.fft2(I)
mag, phase = np.abs(F), np.angle(F)
Fc = np.fft.fftshift(F) # DC -> centre
display = np.log1p(np.abs(Fc))
out = np.fft.ifft2(np.fft.ifftshift(Fc*Hc))
# .real only if conjugate symmetry is valid
```

`fftshift` reorders coefficients, not filtering; undo shift for inverse. Log magnitude is display compression, not a new spectrum. A real spatial filter should preserve conjugate symmetry; a sizeable imaginary inverse signals inconsistency.

6. NumPy shape, axes and safe computation (Lab0)

*: elementwise/broadcast; @: matrix product. Broadcasting compares trailing dimensions: equal or 1; $(H,W,3)/(3,)$ is channelwise. $(N,3)/(N,)$ is usually wrong; row norms need $(N,1)$. `axis` names what is *removed*; `keepdims=True` retains singleton dimensions.

```
Vn = np.linalg.norm(V, axis=1, keepdims=True)
Vunit = np.divide(V, Vn, out=np.zeros_like(V),
                  where=Vn>0) # V is float
chw = hwc.transpose(2,0,1)  # not hwc.T
cols_reversed = A[:, ::-1]
patch = I[y-r:y+r+1, x-r:x+r+1]
rows = A[np.any(A == 8, axis=1)]
```

Slice stop is exclusive; negative starts do not pad. Basic slices are usually views, `copy()` separates storage; advanced indexing usually copies. `reshape` preserves element count/order, does not resize or interpolate; `transpose` permutes axes, .T reverses all axes. `stack` adds an axis; `concatenate` joins an existing one. `argmin/argmax` return first tie by traversal; specify axes/tie rules. `sum` vs `mean`, `norm` axis vs flattened norm are distinct. Retain full precision until final rounding/counts.

FFT linear-convolution recipe, real arrays I and K; output full support, K indexed from array corner:

```
shape = np.array(I.shape) + np.array(K.shape) - 1
J = np.fft.ifft2(np.fft.fft2(I, s=shape)
                 * np.fft.fft2(K, s=shape)).real
# Centred same output, odd K:
ry, rx = np.array(K.shape)//2
Jsame = J[ry:ry+I.shape[0], rx:rx+I.shape[1]]
```

4-point DFT roots: $e^{-2\pi i k/4}$ for k=0,1,2,3 are 1, −i, −1, i. Input [1,0,0,0] gives [1,1,1,1]; [0,1,0,0] gives [1,−i,-1,i] (same magnitude, shifted phase); [a,a,a,a] gives [4a,0,0,0]. Sampling $\cos(2\pi \cdot .75n)$ equals $\cos(2\pi \cdot .25n)$ at integer n: explicit aliasing.

Derivative sign check: centred correlation $[-1, 0, 1]$ on (2,5,9) gives7 (3.5 with factor1/2); convolution gives-7. Constant (5,5,5) gives0. Gradient (3,4) has magnitude5, direction 53.13°, not $\arctan(3/4)$; unsigned orientation ignores simultaneous sign reversal.

1. Edges and Canny

Edge: rapid local intensity change, from material, illumination/shadow, surface orientation or depth discontinuity; not necessarily an object boundary. Texture/noise generate edges; missing/fragmented responses do not automatically form complete curves. Edge tangent is perpendicular to ∇I. Derivative scale selects level of detail.

Canny pipeline: (1) Gaussian smooth/derivative-of-Gaussian filtering; (2) gradient magnitude/direction; (3) directional non-maximum suppression; (4) double threshold and hysteresis linking. Good detection, localization and single/thin response are the design goals. Larger sigma suppresses noise but merges/weaken nearby/fine edges; thresholds set sensitivity.

Directional NMS: compare m to neighbours/interpolated values along *gradient normal*, not edge tangent; retain local maxima. Coarse direction bins often 0,45,90,135 degrees, modulo 180. Specify tie rule. Example: horizontal gradient compares left/right even if the edge itself is vertical.

Hysteresis: high T_H seeds strong edges; low $T_L < T_H$ defines weak candidates. Pixels below T_L rejected; weak pixels survive only through a connected weak/strong path to a strong seed (usually 8-connected). Several weak steps are allowed; isolated weak components vanish. It is not simply keeping every value above low threshold.

Second-derivative method: LoG zero-crossings after smoothing, with strength tests. Difference-of-Gaussians approximates scale-normalized LoG; derivative-of-Gaussian computes spatial derivative. Do not confuse their shared acronym. First derivative gives signed slope; second derivative detects slope change, with stronger noise sensitivity.

2. Corners, second moments and selection (Lab3)

Local shift SSD: $E(u,v) = \sum_w [I(x+u,y+v) - I(x,y)]^2 \simeq (u,v)\,M\,(u,v)^\top$. Structure/second-moment tensor:

$$M = \sum_{x,y\in W} w(x,y) \begin{bmatrix} I_x^2 & I_xI_y \\ I_xI_y & I_y^2 \end{bmatrix} = \begin{bmatrix} a & b \\ b & c \end{bmatrix}.$$

Sum products of gradients, not products of summed/mean gradients. PSD: $a, c \geq 0, ac - b^2 \geq 0$. Eigenvalues give variation along principal directions:

$$\lambda_{1,2} = \frac{a+c \pm \sqrt{(a-c)^2+4b^2}}{2}.$$

Both small = flat/unobservable; one large/one small = edge (ambiguous tangent); both large = corner/textured patch, good 2D localization. Large trace alone does not guarantee a corner. Ill-conditioning $\lambda_{max}/\lambda_{min} \gg 1$ makes estimates unstable.

Harris: $R_H = \det M - k(\operatorname{tr} M)^2 = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2$, often k=0.04–0.06 (use supplied). Large positive = corner-like; negative = edge; small = flat/weak. **Shi–Tomasi:** $R_{ST} = \min(\lambda_1, \lambda_2)$; threshold plus spatial suppression/minimum distance. Used for LK features because both displacement directions need evidence.

Invariance: with isotropic window, rotation transforms M by orthogonal similarity, preserving eigenvalues/determinant/trace. Single-scale Harris is not scale invariant. Under $I' = aI + b$, interior derivatives scale a, M scales a^2, Harris a^4, ST a^2; intensity gain changes numerical thresholds. Spatial NMS picks response-map maxima, distinct from Canny directional NMS and box NMS.

Scale selection: Gaussian scale space $L(x,y,\sigma) = G_\sigma * I$; scale-normalized LoG $\sigma^2\nabla^2 L$ peaks at a characteristic structure size. Extremum over location *and* scale allows corresponding detections after resizing. Small structures can disappear at large scales.

3. Histogram of Oriented Gradients (Seminar 3)

Build/write descriptor: (1) compute G_x, G_y with consistent filters (page2); (2) magnitude $m_p = \sqrt{G_{xp}^2 + G_{yp}^2}$; (3) unsigned $\theta_p = (\text{atan2}(G_{yp}, G_{xp})180/\pi) \bmod 180°$; (4) assign each angle to specified bin; (5) write vector in increasing bin order, including all zeros. For 9 hard bins of width20°, $B_j = [20j, 20(j+1))$, j=0..8, so45° goes to entry j=2 (third). Signed gradients instead span360°.

$$h_j^{count} = \sum_p \mathbf{1}\{\theta_p \in B_j\}, \quad h_j^{mag} = \sum_p m_p \mathbf{1}\{\theta_p \in B_j\}.$$

Unweighted: add1 per pixel. Weighted: add its magnitude. $\sum h^{count}$=number of counted pixels, $\sum h^{mag} = \sum m_p$. Write $h = [h_0, h_1, \ldots, h_8]$; multi-cell descriptor concatenates cell vectors in stated spatial order. These are *raw* histograms; normalize only if requested. Real HOG often interpolates votes and normalizes blocks.

Rotation/sign: basic HOG is not rotation invariant: orientation/spatial bins change; SIFT uses orientation alignment. Flipping both derivative signs preserves m and unsigned orientations but rotates signed angles by 180°. Edge tangent vs gradient normal differs by 90°.

```
mag = np.hypot(Ix, Iy)
ang = np.degrees(np.arctan2(Iy, Ix)) % 180
bins = np.arange(0,181,20)
h_count, _ = np.histogram(ang, bins)
h, _ = np.histogram(ang, bins, weights=mag)
M = np.array([[np.sum(Ix*Ix), np.sum(Ix*Iy)],
              [np.sum(Ix*Iy), np.sum(Iy*Iy)]])
lam = np.linalg.eigvalsh(M) # ascending
```

Corner calculation: $M = \left[\begin{smallmatrix} 16 & 0 \\ 0 & 4 \end{smallmatrix}\right]$ gives eigenvalues 16,4, trace20, det64; k=.04 gives Harris 64 – .04(400) = 48, Shi–Tomasi=4. Edge $M = \left[\begin{smallmatrix} 25 & 0 \\ 0 & 0 \end{smallmatrix}\right]$ gives Harris=-25, ST=0 despite strong trace. M uses squared derivatives, so opposite gradient vectors add evidence rather than cancelling.

S3 Q2d example: the interior patch has

$$G_x = G_y = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \quad m = \begin{bmatrix} 0 & 0 & \sqrt{2} \\ 0 & \sqrt{2} & \sqrt{2} \\ \sqrt{2} & \sqrt{2} & 0 \end{bmatrix}.$$

Five pixels: angle45°, magnitude$\sqrt{2}$; four flat pixels: m=0. Seminar assigns their undefined angle a0° placeholder and counts them in unweighted bin0:

$h^{count} = [4, 0, 5, 0, 0, 0, 0, 0, 0]$,

$h^{mag} = [0, 0, 5\sqrt{2}, 0, 0, 0, 0, 0, 0] \simeq [0, 0, 7.071, 0, 0, 0, 0, 0, 0]$.

If flat pixels are excluded, count bin0 is0 instead4; state this policy. Their weighted vote is always0. Under $I' = aI + b$, a>0, raw magnitudes/votes scale a; interior derivatives remove b. Positive gain leaves unweighted counts unchanged. Normalization can remove common gain; ordinary HOG still lacks rotation invariance.

Canny chain: for low=3, high=8 and adjacent NMS responses 9–5–4–2–6, the 5 and 4 survive through the strong9, the 2 is rejected, and the isolated6 cannot link across2. Hysteresis must run on the thinned response map, not restore a suppressed nonmaximum.

Invariance requires corresponding detected locations/scales, not merely normalized descriptor values; fixed-size patches are not automatically scale invariant.

4. SIFT and local matching (Lab4)

Detect vs describe: detector chooses repeatable locations/scale/orientation; descriptor encodes local appearance to compare. Repeatability is not distinctiveness. Local descriptors allow matches from visible parts under occlusion; geometry/consensus still needed.

SIFT = Scale-Invariant Feature Transform (David Lowe). **S3 Q3a: locate features.** Gaussian scale space $L(x,y,\sigma) = G_\sigma * I$, multiple scales/octave, downsample by2 between octaves. DoG $D = L(k\sigma) - L(\sigma)$ approximates scale-normalized LoG $(k-1)\sigma^2\nabla^2 L$, detecting blobs. Search *spatial/scale extrema*: greater/smaller than all26 neighbours (8 same scale,9 each adjacent scale). Refine (x,y,σ); reject low contrast and unstable edge responses.

Q3b: invariance. (i) Illumination: interior $I' = aI + b$, a>0 gives $D' = aD$: b cancels, gain preserves ideal extremum locations. Gradients remove b; descriptor normalization removes a. **(ii) Scale:** select characteristic σ across scales, sample support proportional to σ; resizing shifts the corresponding scale, not a fixed pixel window. **(iii) Rotation:** Gaussian/magnitude-weighted local orientation histogram (standard36 bins); dominant peak α defines patch-coordinate/gradient alignment. Strong secondary peaks can produce additional oriented keypoints.

Q3c:128 numbers. Aligned 16×16 window →4×4 cells (4×4 samples each),8 gradient-orientation bins/cell over360° (45-degree spacing), *not* raw pixels. Gaussian-weighted magnitude votes split between neighbouring orientation/spatial bins. Concatenate16 cell histograms, e.g. row-major:

$d = [h_{00,0}, \ldots, h_{00,7}, h_{01,0}, \ldots, h_{33,7}] \in \mathbb{R}^{128}$.

Each consecutive group of8 = one subregion's magnitude-weighted orientation histogram. A60° vote splits $2m/3$ to45°, $m/3$ to90°, before other weights. Relative angle $(\theta - \alpha) \bmod 360°$ distinguishes opposite gradients. L2-normalize, clip entries (standard.2), renormalize. Contrast cutoffs, clipping, nonuniform lighting, large projective/deformation changes limit exact invariance.

Matching: nearest descriptor distance under metric (often L2 for SIFT); brute force n query by m database descriptors of D dimensions costs $O(nmD)$. Nearest absolute distance alone does not show distinctiveness in repeated texture. Ratio test with nearest d_1, second-nearest $d_2 > 0$: keep if $d_1/d_2 < \tau$. Near 1 = ambiguous. τ is a chosen threshold, not guaranteed correctness. If squared distances $D_i = d_i^2$, compare $D_1/D_2 < \tau^2$, not τ. Mutual nearest/cross-check tests a different consistency rule.

Geometric verification: estimate affine/H/F from matches with RANSAC, count residual-compatible inliers, refit. Descriptor match ≠ physically correct match. Local-keypoint retrieval can survive partial occlusion; histograms/global embeddings can lose layout. SURF = Speeded-Up Robust Features; ORB = Oriented FAST and Rotated BRIEF (named alternatives, not advanced implementation requirements).

5. Least squares and robust line/shape fitting

OLS: $\min_\theta \|A\theta - y\|^2$; full-column-rank solution $(A^\top A)^{-1}A^\top y$, stable implementation `lstsq`. Rank deficiency gives nonunique solutions; numerical routine may return a minimum-norm fit, not evidence of unique geometry. Normal equations square condition number. An eigenvector/SVD nullspace solves *homogeneous* fitting (page 4), not ordinary $A\theta = y$.

Line y=mx+b: rows $[x_i, 1]$, unknown (m, b); vertical y residuals, unsuitable for vertical lines/symmetric x,y noise. For $ax+by+c=0$, perpendicular residual $(ax_i+by_i+c)/\sqrt{a^2+b^2}$; normalize $a^2+b^2=1$. Total LS: subtract point centroid; smallest-eigenvalue eigenvector of scatter matrix gives normal (a,b); $c = -a\bar{x} - b\bar{y}$. Vertical line has b=0.

Loss choice: scalar constant minimizing squared error = mean; absolute error = median (even count: any point between middle two is optimal). L2 gives large outliers quadratic influence. Huber $\ell_\delta(r) = r^2/2$ for $|r| \leq \delta$, else $\delta(|r| - \delta/2)$: quadratic near 0, linear tails. Robustness does not remove all ambiguity.

Hough (Seminar 2): each edge point votes for line parameters; accumulator peaks identify supported lines. Normal form $\rho = x\cos\theta + y\sin\theta$ handles vertical lines with bounded θ, unlike y=ax+b's infinite slope. In slope/intercept space each point votes on $b = y - ax$; intersections indicate common lines. Bin resolution, noise and multiple peaks affect detection. Hough votes in parameter space; RANSAC samples models and tests consensus.

6. RANSAC = RANdom SAmple Consensus

Sample minimal nondegenerate subset → fit → residuals for all data → threshold/count inliers → retain best → refit using consensus. Choose model, residual units, threshold, iteration budget and score/tie rule. Inliers are threshold-compatible, not guaranteed true matches. Larger threshold increases count but can bias fit; zero threshold is unrealistic with noise.

Minimal sample: line 2 distinct; circle 3 noncollinear; 3D plane 3 noncollinear; 2D affine 3 noncollinear source pairs; homography 4 pairs in general position (no three collinear in minimal set). Duplicate/-collinear/dependent geometry can make all-inlier samples unusable.

$$q = w^s, \quad p_N = 1 - (1-q)^N, \quad N = \left\lceil \frac{\log(1-p)}{\log(1-w^s)} \right\rceil.$$

w = inlier fraction; s = sample size; p = desired success. Lower w/higher s/higher p requires more iterations. Use ceiling, $0 < q < 1$; q=0 impossible, q=1 one trial if nondegenerate. With distinct draws within a finite sample, exact $q = \binom{I}{s}/\binom{N}{s}$ for I inliers/N points (rename iteration count to avoid confusion).

Guarantees: random resampling can repeat bad subsets indefinitely; finite budget gives probability, not certainty. If enumerating every distinct subset without repetition, worst-case first all-inlier sample occurs by $\binom{N}{s} - \binom{I}{s} + 1$ (assuming valid all-inlier sets). No universal 50%-inlier requirement. Competing structures can dominate; refitting one consensus does not solve arbitrary multi-model fitting.

TLS and iteration-budget code:

```
mu = pts.mean(axis=0)     # N x 2 points
_, _, Vt = np.linalg.svd(pts-mu)
a, b = Vt[-1]             # unit line normal
c = -np.dot([a,b], mu)
import math
trials = math.ceil(math.log1p(-p)
                   / math.log1p(-(w**s)))
```

With w=.5, s=4, target p=.99, trials=72 (not71); with s=2 it is17. These budgets presume independent trials and nondegenerate all-inlier samples. A ratio test with distances10,12 and $\tau = .8$ rejects (10/12> .8); squared distances100,144 also reject when compared to .64. Equal closest distances remain ambiguous even if both tiny.

Plane: $ax+by+cz+d=0$, distance $|ax+by+cz+d|/\sqrt{a^2+b^2+c^2}$. Cross two sample displacements for its normal; parallel vectors are degenerate.

7. Optical flow and the aperture problem

Optical flow: apparent image-plane displacement/velocity (u,v), not directly metric 3D motion. True projected motion field and measured flow can differ: lighting changes yield apparent motion; uniform rotating sphere can have physical motion but no visible change. Camera and object motion, depth and projection all affect flow.

Brightness constancy: $I(x+u, y+v, t+1) = I(x,y,t)$. First-order Taylor expansion for small motion:

$I_x u + I_y v + I_t = 0$.

If u,v are displacement/frame, I_t is change over that frame step; physical-time derivatives give velocity units. One equation/two unknowns: constrains normal motion only. Minimum-norm normal flow $-I_t \nabla I / \|\nabla I\|^2$; any tangent component remains free. Straight edge observed through a small aperture cannot reveal motion along itself.

8. Lucas–Kanade estimation (Lab3)

Assume one constant flow vector in local W; stack $[I_x, I_y]$ rows into $A \in \mathbb{R}^{n\times 2}$, $b = -I_t$:

$$\hat{f} = \arg\min_f \|Af - b\|^2, \quad (A^\top A)\hat{f} = A^\top b.$$

Weighted windows: $M = A^\top W A$, $b' = -\sum w[I_x, I_y]^\top I_t$. With $a = \sum wI_x^2, b = \sum wI_xI_y, c = \sum wI_y^2, d = -\sum wI_xI_t, e = -\sum wI_yI_t$:

$$u = \frac{cd - be}{ac - b^2}, \quad v = \frac{ae - bd}{ac - b^2}.$$

Requires rank 2, both eigenvalues sufficiently large, reasonable condition number. Flat patch: both small/no evidence. Edge: rank≈ 1/tangent ambiguity. Corner: both directions constrained. Same tensor as corner detection. Larger window adds evidence but may combine different motions/depths and violate constant-flow assumption.

Assumptions/failures: brightness constancy; small displacement for linearization; local constant motion; sufficient texture; visible correspondence. Lighting/shadows/specular changes, occlusion/disocclusion, repetitive patterns, large motion, motion boundaries and changing shape can fail. Occluded points have no matching measurement. A stable least-squares result alone does not prove those assumptions.

```
A = np.column_stack((Ix.ravel(), Iy.ravel()))
b = -It.ravel()
u, v = np.linalg.lstsq(A, b, rcond=None)[0]
```

Coarse to fine: antialias/downsample images; estimate at coarsest scale; upsample vector field and multiply vector values by scale ratio; warp second frame with current flow; solve small residual increment; update/refine at each finer scale. Half-resolution displacement (u_c, v_c) becomes $(2u_c, 2v_c)$ at full resolution, *not* just a spatial resize. For downscale factor a, full displacement = a×coarse displacement.

Iterative warp: compare reference $I_1(x,y)$ to $I_2(x+u, y+v)$, then estimate residual; sign depends on which frame is sampled. Pyramids permit larger motion but small/thin features can vanish; bad coarse matches can trap fine refinement. Border/occlusion masks needed.

Sparse vs dense: sparse estimates selected keypoints (fast, useful for tracks); dense estimates most/all pixels (richer field, greater cost/ambiguity in textureless regions). Horn–Schunck uses global smoothness for dense flow; LK uses local constant-motion constraints (advanced HS code optional).

Applications: stabilization estimates/removes global camera motion; interpolation warps frames toward intermediate times (fails at occlusions/new content); traffic speed needs calibration, depth/road geometry, timing–pixels/frame alone is not km/h. Known camera translation/baseline plus parallax can constrain depth; flow alone does not set metric scale. Also motion segmentation, compression, denoising/super-resolution.

9. Template, detection and mean-shift tracking

Template tracking: search near previous position, minimize $\text{SSD}(d) = \sum_x [I(x+d) - T(x)]^2$, or maximize $\text{CC}(d) = \sum_x I(x+d)T(x)$. $\text{SSD} = \|I\|^2 + \|T\|^2 - 2\text{CC}$; maximizing CC equals minimizing SSD only when candidate energy is constant. SSD is sensitive to offsets/gain; raw CC can favour bright/high-energy patches.

$$\text{ZNCC} = \frac{\sum (I - \bar{I})(T - \bar{T})}{\sqrt{\sum (I-\bar{I})^2 \sum (T-\bar{T})^2}}.$$

For nonzero variance, mean subtraction removes additive offset; normalization removes positive gain. Non-centred normalized correlation does not remove offsets. Uniform patches make normalization undefined. Limited search = motion prior/computation saving, but can miss fast motion; wide search raises ambiguity.

Tracking by detection: classify candidate regions each frame, associate with appearance/motion, update target model. Drift: wrong self-labelled updates progressively learn background/occluder. Conservative updates and reliable visible features can help, not guarantee recovery. Fixed template avoids update drift but cannot adapt well to appearance/pose change.

Mean-shift tracking: move candidate window toward increased colour-histogram similarity to target; local mode-seeking, not exhaustive search. Histogram tolerates some pixel rearrangement/partial occlusion but loses spatial layout; similarly coloured background, large jumps and lighting/scale changes can fail. Colour similarity can use Bhattacharyya coefficient $\sum_b \sqrt{p_b q_b}$; bandwidth/window sets scale. Local feature tracks can survive occlusion if enough visible, geometrically consistent matches remain.

Flow units/example: coarse flow (1.5,-.5) at one-quarter resolution means full displacement (6,-2) pixels/frame. At 25 frames/s, horizontal speed=150 pixels/s, not150 metres/s. Converting it to physical speed needs calibrated scene geometry; changing frame spacing changes displacement even for the same physical velocity. If patch equations are $2u = 4$ and $3v = -3$, LK gives (2,-1). Replacing the second by $4u = 8$ leaves v unconstrained despite two observations: rank matters, not equation count.

1. Global 2D transforms and invariants

One mapping across image, unlike independent per-pixel flow. With column homogeneous coordinates $\tilde{x}' \sim H\tilde{x}$:

$$T=\begin{bmatrix}1&0&t_x\\0&1&t_y\\0&0&1\end{bmatrix},\quad S=\mathrm{diag}(s_x,s_y,1),\quad R=\begin{bmatrix}c&-s&0\\s&c&0\\0&0&1\end{bmatrix}.$$

Translation 2 DoF; **rigid/Euclidean** $x'=Rx+t$, 3 DoF, preserves distances/angles/area; **similarity** $x'=sRx+t$, 4 DoF, preserves angles and length ratios, changes absolute lengths/area. Reflection is not a proper rotation. Nonuniform scaling/shear can change angles.

Affine $x'=Ax+t$, H last row $[0,0,1]$, 6 DoF; nonsingular A preserves lines, collinearity, parallelism, and ratios *along one line*; not general lengths/angles. Areas scale by $|\det A|$, not necessarily unchanged.

Projective/homography: nonsingular 3×3 H up to scale, 8 DoF, preserves incidence/lines/collinearity (and cross-ratios), generally not parallelism, angles, distance ratios.

$$x'=\frac{h_{11}x+h_{12}y+h_{13}}{h_{31}x+h_{32}y+h_{33}},\quad y'=\frac{h_{21}x+h_{22}y+h_{23}}{h_{31}x+h_{32}y+h_{33}}.$$

Denominator zero maps to infinity. H and αH equivalent, $\alpha\neq 0$; can set $h_{33}=1$ only when nonzero. H followed by T = TH, not HT; inverse map uses H^{-1}. General line transforms as $\ell' \sim H^{-\mathsf{T}}\ell$. In 3D use 4×4 translation/rotation/scale; rotate→translate→scale = STR.

2. Estimating affine transforms and homographies

Affine: unknown (a,b,t_x,c,d,t_y); each pair $(x,y)\leftrightarrow(u,v)$ gives rows $[x,y,1,0,0,0]$ with target u and $[0,0,0,x,y,1]$ with v. Stack $2N\times 6$ LS system. Minimum 3 noncollinear source pairs; use extra pairs for noise, RANSAC for outliers.

Homography DLT: vector h = row-major entries of H; each pair gives: $\begin{bmatrix}x&y&1&0&0&0&-ux&-uy&-u\\0&0&0&x&y&1&-vx&-vy&-v\end{bmatrix}h=0$.

Stack A shape $2N\times 9$. Minimum 4 pairs in general position. $\min\|Ah\|^2$ alone gives invalid h=0; enforce $\|h\|=1$. Smallest eigenvector of $A^{\mathsf{T}}A$ / last *full* right singular vector of A solves homogeneous LS. Normalize coordinates for numerical stability (centroid 0, mean distance $\sqrt{2}$), fit H_n, denormalize $H=T_2^{-1}H_nT_1$. SVD nullspace differs from `lstsq(A,0)`.

```
_, _, Vt = np.linalg.svd(A, full_matrices=True)
H = Vt[-1].reshape(3,3) # 4-pair A is 8x9
q = H @ np.array([x,y,1.])
uv = q[:2] / q[2]
```

Residual: one-way reprojection $\|x_i'-\pi(H\tilde{x}_i)\|$, $\pi(a,b,w)=(a/w,b/w)$; symmetric adds reverse direction. Algebraic DLT residual is not pixel distance. RANSAC selects inliers using declared residual/units, then refit and optionally minimize geometric error.

3. Warping, interpolation and stitching (Lab4)

Forward: map source grid points to destination and splat; rounding can make holes/collisions. **Backward:** for every output point q, evaluate $I_{out}(q)=I_{src}(\pi(H^{-1}\tilde{q}))$; sample source by interpolation. Fills covered destination grid, not invalid/outside coverage. Anti-alias before strong minification.

Bilinear: source x=i+a, y=j+b, $0\le a,b<1$; weights TL $(1-a)(1-b)$, TR $a(1-b)$, BL $(1-a)b$, BR ab; sum 1, access $I[j,i]$. Nearest uses nearest pixel (blocky); bilinear is linear in each coordinate, not higher-order image reconstruction. Use specified out-of-domain rule.

One H exists for one scene plane (between suitable ideal cameras), or pure rotation about same optical centre: $H=K_2R_{21}K_1^{-1}$. Translation through nonplanar scene gives depth-dependent parallax; one H cannot align all depths. Undistort first; camera-centre rotation differs from rotating about some external pivot.

Image-stitching steps (S4 Q1b): combine overlapping images into one aligned panorama; same optical centre implies rotation homography. **1. Detect/describe:** repeatable corners/blobs plus local appearance vectors; examples *SIFT, ORB* (Harris detects corners but still needs a descriptor). **2. Match:** compare descriptors across images, reject ambiguous correspondences; examples *brute-force L2 + Lowe ratio* for SIFT, *Hamming-distance matching* for binary ORB (cosine is another declared similarity). **3. Estimate alignment:** fit H between corresponding coordinates, reject outliers, refit on all inliers; examples *DLT, RANSAC*. Minimum4 generic pairs. Lab4's *affine LS + RANSAC* is a simpler6-DoF model; use H for general same-centre stitching, $H=K_2R_{21}K_1^{-1}$. **4. Planar map/warp:** transform image corners to choose common canvas, translate negative bounds, resample each view into it; examples *backward mapping + bilinear* or *nearest-neighbour* interpolation. Source-to-canvas $T_{canvas}H$; keep coverage masks (real black pixels ≠ missing data). **5. Blend:** combine aligned overlap and copy singly covered pixels; examples *feathering/alpha blending* (normalized spatial ramp weights), *multiband/pyramid blending* (blend across scales to soften seams).

Blending: $I=(w_1I_1+w_2I_2)/(w_1+w_2)$ where covered, or $\alpha I_1+(1-\alpha)I_2$. Distance/ramp weights soften seams; exposure compensation can reduce brightness mismatch. Misalignment/parallax/moving objects still produce ghosting; blending cannot repair geometry. Outside overlap copy valid source rather than average with zero fill.

4. Calibration and lens distortion

Known pattern/object coordinates and image observations estimate K, distortion and poses. Intrinsics describe sensor/lens mapping; extrinsics depend on reference frame/view. Planar target images from different poses can calibrate a camera; 2D image-to-image H is not a complete arbitrary-scene 3D reconstruction.

Typical distortion model in normalized x,y with $r^2=x^2+y^2$: radial factor $a=1+k_1r^2+k_2r^4+\cdots$, tangential p_1,p_2:

$$x_d=ax+2p_1xy+p_2(r^2+2x^2),$$
$$y_d=ay+p_1(r^2+2y^2)+2p_2xy.$$

Coefficients/signs depend on model. Apply K after normalized-coordinate distortion; undistortion solves the reverse, not simply applying the same polynomial. Higher-order distortion is outside simple pinhole P.

Warp/sample code: already computed source coordinates xs,ys must be valid for chosen interpolation; don't confuse x with row index.

```
ix, iy = np.floor(xs).astype(int), np.floor(ys).astype(
    int)
a, b = xs-ix, ys-iy
val = ((1-a)*(1-b)*I[iy,ix] + a*(1-b)*I[iy,ix+1]
     + (1-a)*b*I[iy+1,ix] + a*b*I[iy+1,ix+1])
```

For corner values TL=0, TR=10, BL=20, BR=30 and a=.25,b=.5, weights=(.375,.125,.375,.125), interpolated value=12.5. At a=b=0, returns TL. At image last row/column, use specified edge rule–blindly accessing +1 is invalid.

5. Two-view geometry: rays, planes and epipoles

Camera centres C_1, C_2 and scene point X define epipolar plane; intersections with image planes are epipolar lines. Epipole e = other camera centre's image (possibly at infinity). All epipolar lines in one view pass through its epipole. Point in view 1 restricts correspondence in view 2 from 2D search to a 1D line, not a unique match.

Fundamental matrix on pixel homogeneous points:

$$\tilde{x}_2^{\mathsf{T}}F\tilde{x}_1=0,\quad \ell_2=F\tilde{x}_1,\quad \ell_1=F^{\mathsf{T}}\tilde{x}_2.$$

F: 3×3, rank 2, 7 DoF = 9 entries minus common scale minus rank constraint. $Fe_1=0$, $F^{\mathsf{T}}e_2=0$. **Rank is from epipolar null direction, not scale ambiguity**: identity up to scale is still rank 3. For line $\ell=(a,b,c)$, point distance $|ax+by+c|/\sqrt{a^2+b^2}$; raw algebraic residual depends on line scale.

Essential matrix on calibrated normalized coordinates $y_i=K_i^{-1}\tilde{x}_i$, relative pose $X_2=RX_1+t$:

$$E=[t]_\times R,\quad y_2^{\mathsf{T}}Ey_1=0,\quad F=K_2^{-\mathsf{T}}EK_1^{-1}.$$
$$[t]_\times=\begin{bmatrix}0&-t_z&t_y\\t_z&0&-t_x\\-t_y&t_x&0\end{bmatrix},\quad E=K_2^{\mathsf{T}}FK_1.$$

E: rank 2, two equal nonzero singular values and one 0, 5 DoF = rotation 3 + translation *direction* 2. Epipolar geometry does not recover translation magnitude. E and F are not interchangeable without intrinsics.

Estimation overview: F linear 8-point algorithm (not the same as 7 DoF); each $\tilde{x}_2^{\mathsf{T}}F\tilde{x}_1=0$ is one equation. Normalize points, solve homogeneous system, enforce rank 2 by SVD setting last singular value 0, denormalize. E can use calibrated 5-point algorithm; advanced derivation not required here. RANSAC excludes mismatches before pose/reconstruction. Pure rotation has t=0, E=0 and no ordinary triangulation baseline; use rotation homography, depth remains unknown.

6. Rectification, disparity and depth

Rectification: warp views so corresponding epipolar lines become horizontal rows, often with common calibration/principal-point convention. This simplifies matching, not making images identical; occlusion and horizontal disparity remain. Matched v coordinates agree in ideal rectified case.

Declared convention: parallel cameras, right centre translated +B along left-camera x, f in pixels, equal c_x; $d=u_L-u_R$:

$$u_L=c_x+fX/Z,\quad u_R=c_x+f(X-B)/Z,$$
$$d=fB/Z,\quad Z=fB/d,$$
$$X=(u_L-c_x)Z/f_x,\quad Y=(v_L-c_y)Z/f_y.$$

If $c_{xL}\neq c_{xR}$, use corrected $d=(u_L-c_{xL})-(u_R-c_{xR})$. HC6b prints opposite disparity sign in one diagram; never copy sign without camera convention. Baseline/depth share physical units; f,d both pixels. Near objects larger d; infinite depth d→0. Negative d under this setup suggests sign mismatch/bad correspondence (or incompatible camera setup).

Sensitivity: $\partial Z/\partial d=-fB/d^2$; $\sigma_Z\simeq(fB/d^2)\sigma_d$, relative $\sigma_Z/Z\simeq\sigma_d/|d|$. Greater f/B increases disparity for fixed depth and can improve depth precision; larger baseline also changes visibility/appearance. No baseline gives no metric depth from parallax.

Matching: evaluate $C(x,y,d)$ along epipolar row; local SSD $\sum_W[I_L(x,y)-I_R(x-d,y)]^2$, SAD $\sum_W|\cdot|$, or correlation/ZNCC. Winner-take-all $\arg\min_d C$ can be noisy. Small windows preserve boundaries but ambiguous; large windows reduce noise but mix depths/oversmooth edges. Textureless/repeated regions, specularity, illumination differences and occlusion break simple matching.

Regularized stereo:

$$E(D)=\sum_p C(p,D_p)+\lambda\sum_{(p,q)}V(D_p,D_q).$$

Data term measures appearance consistency; smoothness encourages nearby compatible disparities (e.g. $|D_p-D_q|$, truncated penalties). Large λ can erase true depth jumps; $\lambda=0$ allows noisy independent choices. Left–right consistency can reject occluded/bad matches; filling holes is inference, not measured correspondence.

Triangulation: intersect backprojected rays from different centres; with noise minimize reprojection/ray discrepancy. For known camera P and observed (u,v), homogeneous point obeys $(uP_3-P_1)X_h=0$, $(vP_3-P_2)X_h=0$; stack views, solve constrained nullspace, dehomogenize. Rays nearly parallel give poor depth conditioning. Cheirality checks positive depth in visible cameras.

Fast sanity checks: projected scale $\propto f/Z$; camera at C means $RC+t=0$; proper R preserves vector norm; H maps one point to one point, F maps one point to a *line*; rectify before using row-only disparity formula; small disparity means large/error-sensitive depth; pure rotation cannot triangulate scale/depth.

Unit recipes: stereo $f=800$ px, B=0.10 m, d=20 px ⇒ $Z=4$ m. Doubling d halves Z. At image downscale a, $f'=f/a, d'=d/a$ leaves $f'B/d'=Z$. Cropping changes principal points; neglecting unequal crops can bias d and Z. A finite homography point $(6,9,3)$ is $(2,3)$, not $(6,9)$. Always divide once at final projective step.

Epipolar/depth computation:

```
F = np.linalg.solve(K2.T, E) @ np.linalg.inv(K1)
l2 = F @ np.array([u1,v1,1.])
dist = abs(l2 @ [u2,v2,1.]) / np.hypot(l2[0],l2[1])
Z = f * B / disparity     # valid rectified convention
sigma_Z = abs(f*B / disparity**2) * sigma_d
```

At f=800,B=.1,d=20, $\sigma_d=.5$ pixel yields $\sigma_Z\simeq.1$ m (2.5% of Z=4 m). Halving disparity doubles Z and quadruples absolute depth uncertainty at fixed σ_d. Reversing F's point order requires transpose; multiplying F by any nonzero scalar leaves distance after absolute normalization unchanged.

Multi-view counts: M calibrated poses have 6M unknowns, N 3D points 3N; each point/view observation gives 2 pixel residuals, not3. A connected calibrated monocular model has 7-DoF global similarity gauge. More equations than unknowns does not ensure rank/uniqueness; one-view/unobserved points lack triangulation evidence.

7. Structure from Motion and multi-view stereo

SfM = Structure from Motion: jointly recover camera poses and mostly sparse static-scene 3D points from overlapping images. Unknown extrinsics/structure vs known-camera stereo. Moving objects violate common rigid-scene explanation. **MVS = Multi-View Stereo:** densify using estimated/calibrated cameras and multiple observations; usually after SfM.

Incremental pipeline: extract repeatable descriptors → match images/build tracks → geometrically verify with E/F/H RANSAC → choose well-conditioned initial pair → recover relative pose/triangulate → register new cameras via 2D–3D PnP → triangulate new points → filter outliers/refine → repeat. Good initialization requires overlap/texture and useful baseline, not pure rotation. Track = same scene point seen across images, not unrelated pairwise matches.

PnP = Perspective-n-Point: known 3D points + their observed 2D locations + intrinsics → camera pose. Unlike two-view E/F fit (2D–2D) or triangulation (known cameras, unknown 3D point). Outliers and degenerate geometry still need care. Pose changes require transforming world points before projection.

Bundle adjustment: joint nonlinear camera/point refinement minimizes summed reprojection error:

$$\min_{\{K_i,R_i,t_i,X_j\}}\sum_{(i,j)\in O}\|x_{ij}-\pi(K_i(R_iX_j+t_i))\|^2.$$

May use robust residual loss; optimize only specified camera parameters. O contains available visible observations; not every camera sees every point. Gauge/reference constraints required to prevent arbitrary global frame/scale changes. BA is not just independent point fitting with fixed cameras, and minimizes image error rather than a supplied ground-truth 3D distance.

Ambiguity: calibrated monocular reconstruction is defined up to global similarity (rotation, translation, positive scale); cannot call units metres without metric reference. Known baseline/object dimension sets scale. Uncalibrated reconstruction can have larger projective ambiguity. Scale ambiguity in SfM and homogeneous scale of individual matrices/points are related bookkeeping issues, not proof that every matrix is singular.

Global vs hierarchical: global estimates multi-camera relations jointly; incremental adds views one at a time; hierarchical reconstructs subgroups and merges, often by similarity transforms. All need correct image associations, outlier handling and connected observation coverage. More views can improve coverage/noise, but repeated texture and systematic errors remain.

Sparse vs dense: sparse keypoints give a skeleton/poses; MVS recovers surfaces/depth maps where photometric consistency and visibility support them. Occluded/textureless/reflective surfaces remain hard. Fusion merges compatible multi-view estimates; it cannot turn invisible geometry into a direct observation. COLMAP/OpenMVG/Photo Tourism are example SfM systems, not required advanced implementations.

8. Shape from X: cues and assumptions

Shading: normal-related intensity, needs assumptions on material/light/camera; convex/concave and light-direction ambiguities. **Photometric stereo:** fixed view/different known lights, albedo and normals then integration (page 1). **Geometric stereo/motion:** different centres, matching/parallax then depth. Don't use varying-light equations for moving-camera data without aligning geometry.

Texture: assumed repetitive surface pattern; density, scale, distortion and foreshortening reveal orientation/depth variation. Unknown texture can be mistaken for geometry. **Focus/defocus:** sweep focus; highest local sharpness/high-frequency response relates to depth through calibrated optics. Textureless regions offer poor focus cues. **Specularities:** view-dependent reflection constrains surface orientation under known/constrained illumination; diffuse assumptions fail.

Shadows: shadow geometry gives shape/height constraints with light and viewing geometry. Shadow edge is not necessarily a surface/material edge. **Silhouettes:** known cameras define viewing cones; intersection is visual hull. Concavities that never alter silhouettes cannot generally be recovered; silhouettes give boundary/occupancy constraints, not complete internal surface detail. SIRFS = Shape, Illumination and Reflectance from Shading; jointly estimating unknown components needs stronger priors than basic known-light SfS.

What geometry cannot promise: correct matrix dimensions alone do not establish calibration, unique solution, accurate match, visibility or metric scale. Check rank, residual definition, conditioning, frame/sign, masks and model applicability before arithmetic.

Multi-view algebra and reconstruction recipes

Global scale proof: for any a>0, replace $X_j\to aX_j, t_i\to at_i$, leave R,K unchanged. Then $K_i(R_i(aX_j)+at_i)=aK_i(R_iX_j+t_i)$; perspective division cancels a. Therefore all pixels/reprojection errors unchanged while all scene/camera distances scale a. Baseline or known physical dimension removes this ambiguity.

Two-camera triangulation from pixel coordinates and known P1,P2:

```
A = np.stack([u1*P1[2]-P1[0], v1*P1[2]-P1[1],
              u2*P2[2]-P2[0], v2*P2[2]-P2[1]])
_, _, Vt = np.linalg.svd(A)
Xh = Vt[-1]
X = Xh[:3] / Xh[3]       # only if finite
# Reproject to check pixel residuals and depth
q = P1 @ np.r_[X,1.]
err = np.linalg.norm(q[:2]/q[2] - [u1,v1])
```

For f=800, B=.1 m, $(c_x,c_y)=(320,240)$, left pixel (400,260), right (380,260): d=20, Z=4 m, X=.4 m, Y=.1 m. With same point viewed by right camera, camera-frame x=.3 m; reproject right u=380. A two-pixel/three-pixel residual has Euclidean magnitude $\sqrt{13}$, squared contribution13–don't mix error with squared error in thresholds.

Low parallax: good homography alignment need not constrain depth. Pure rotation has no baseline; more near-identical views do not repair this.

1. Tasks, representations and variation

Classification: label whole image (single/multilabel). **Localization**: class + position of a target, often one box. **Detection**: classes and boxes of multiple instances. **Retrieval**: rank database images relevant to query. **Semantic segmentation**: class per pixel, no same-class instance separation. **Instance**: distinct object masks; **panoptic**: semantic labels for all pixels plus instance IDs for countable "things", including "stuff" like sky.

Semantic gap: low-level pixels/features vs high-level meaning. Challenges: viewpoint, illumination, scale, deformation, clutter, occlusion, intraclass variation, interclass similarity. Low-level = colour/edges; mid-level = motion/depth; high-level = recognition/semantic interpretation. SLAM = Simultaneous Localization and Mapping; SAM = Segment Anything Model (intro applications, not advanced architecture requirement).

2. Clustering: k-means vs mean shift

k-means: unsupervised, choose K centres; minimize $J = \sum_i \|x_i - \mu_{z_i}\|_2^2$.

Initialize centres; alternate $z_i = \arg\min_k \|x_i - \mu_k\|^2$ and $\mu_k = (1/n_k)\sum_{z_i=k} x_i$; stop assignments/objective/iteration cap. Each exact step cannot increase J; local optimum, not necessarily global. K, feature scaling, metric, initial seeds, empty-cluster rule and outliers matter; multiple starts help. Mean minimizes fixed-cluster squared error.

Image clustering: colour-only features can assign disconnected distant pixels same cluster; cluster ID ≠ semantic object/class ≠ connected component. Adding $(\alpha x, \alpha y)$ encourages spatial compactness; relative units/α set influence. Large K can fragment, small K merge; no universal choice.

Mean shift: mode-seeking kernel-density procedure; update location to local weighted mean:

$$x^{new} = \frac{\sum_i w_i(x)\, x_i}{\sum_i w_i(x)}, \quad w_i = g(\|(x - x_i)/h\|^2).$$

Move repeatedly until convergence; initial positions attracted to same mode share basin/cluster (with mode-merging rule). No preset number of clusters in basic form; bandwidth h controls scale, small h often many fine modes, large h merges. Kernel/feature scale changes behaviour. Mean shift seeks density modes, k-means seeks squared-distance centres; k-NN predicts using labelled neighbours.

3. Bag-of-Visual-Words (BoVW/BoW)

Training descriptors (SIFT or sampled patches) → k-means vocabulary $\{\mu_k\}_{k=1}^K$ → nearest-word quantization → K-bin image histogram → normalize → classifier/retrieval similarity. Hard count $h_k = \sum_j \mathbf{1}\{\arg\min_l \|d_j - \mu_l\| = k\}$; L1 $h/\sum h$, L2 $h/\|h\|$. K sets representation dimension, not descriptor count/image size.

Trade-offs: loses layout and quantization detail, vocabulary ambiguity/repeated texture; spatial grids can retain coarse layout if explicitly used. Dense sampling = regular coverage; interest points = salient repeatable features. Which wins depends on distinctiveness/coverage/cost/dataset, not a universal database-size law. Fit codebook and preprocessing on train only; validate K/normalization/classifier, keep test untouched.

4. k-NN, linear models and SVM

k-NN: supervised, store labelled training vectors, find k nearest, majority/weighted vote with explicit tie rule. $d_1 = \sum_j |x_j - y_j|$, $d_2 = \sqrt{\sum_j (x_j - y_j)^2}$; squared L2 gives same nearest ranking. Feature scaling matters. k=1 sensitive to noise/overfit; larger k smoother, may underfit. Naive query $O(ND)$ distance work; little fitting but large prediction/storage cost. Validate k/metric, do not tune on test.

Linear scores: $s = Wx + b$, $W \in \mathbb{R}^{C\times D}$; predict $\arg\max_c s_c$. Pairwise class boundaries are hyperplanes in chosen feature space. One-vs-rest trains C binary classifiers; nonlinear features can yield nonlinear raw-pixel boundaries.

SVM = Support Vector Machine: maximize margin with norm regularization; binary y=±1, hinge $\max(0, 1 - y(w^\top x + b))$. Soft-margin objective $\frac{1}{2}\|w\|^2 + C\sum_i \max(0, 1 - y_i s_i)$. Larger C prioritizes violations/fit, smaller C stronger relative regularization; margin boundaries $s = \pm 1$, width $2/\|w\|$. Support vectors have nonzero dual weights; can lie on/inside margin or be misclassified. Kernel trick computes implicit-feature inner products (e.g. nonlinear boundary) without explicit mapping.

Splits: train fits parameters; validation chooses hyperparameters/architecture/early stopping; final test estimates selected pipeline. Cross-validation rotates training/validation folds, not repeated test tuning. Standardization, vocabularies and augmentation must not leak held-out information. Parameters are learned weights; hyperparameters are selected settings (K,k,rate,etc.).

Vector quantization and classifier calculations

```
# X: N x D features; centres: K x D
dist2 = ((X[:,None,:]-centres[None,:,:])**2).sum(-1)
labels = dist2.argmin(axis=1)
new = centres.copy()
for k in range(len(centres)):
    if np.any(labels == k):
        new[k] = X[labels == k].mean(axis=0)
# Here empty clusters keep old centre (chosen policy)
hist = np.bincount(labels, minlength=len(centres))
```

Same nearest-centre assignment forms hard visual words; a codebook learned once on training descriptors is reused, not re-fit per test image. Histogram (2,0,1) has L1-normalized (2/3,0,1/3), L2-normalized $(2,0,1)/\sqrt{5}$. Normalization scale changes similarities even though argmax bin remains.

k-NN example: ordered neighbour labels A,B,B: k=1 predicts A, k=3 predicts B by unweighted majority. Inverse-distance weighting could give a different answer; handle zero distance/ties explicitly. Euclidean distance to $(0,0)$ from (3,4) is5; L1 is7. Distances in different feature units should be scaled from training statistics.

Margin example: w=(3,4) has norm5, hard-margin width .4 under $s = \pm 1$ normalization. Large raw scores alone do not imply a larger geometric margin; weight norm also matters.

5. Classification metrics and confusion matrices

For chosen positive class: TP actual+/pred+; FP actual-/pred+; FN actual+/pred-; TN actual-/pred-. With confusion rows actual and columns predicted, $TP_c = C_{cc}$, $FN_c = \sum_j C_{cj} - TP_c$, $FP_c = \sum_i C_{ic} - TP_c$; TN = total minus others. Check orientation before sums.

$$\text{Acc} = \frac{TP+TN}{TP+TN+FP+FN},$$

$$P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}.$$

$$F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}.$$

Recall = sensitivity/TPR; specificity = $TN/(TN+FP)$, FPR = $FP/(FP+TN)$. Accuracy can look high with zero minority recall. F1 is harmonic, not arithmetic mean; one weak P/R hurts. Define zero-denominator rule; do not silently claim 0 or 1. Percent vs fraction changes factor 100.

Averages: macro = equal mean of class-specific metrics; weighted macro weights by specified supports; micro = pool TP/FP/FN then compute. Complete single-label multiclass micro P/R/F1 = accuracy. Macro F1 ≠ F1 of macro P,R in general. Top-k accuracy = true label among k highest scores; top-5 is not five predictions scored independently.

Threshold trade-off: fixed score ordering, lower cutoff gives more predictions and recall cannot decrease; precision need not be monotonic. Precision–recall curve varies cutoff/rank. Class imbalance affects precision and makes accuracy less useful; score calibration is different from ranking quality.

6. Retrieval, AP and mAP

Ranking: first K results, T relevant, G total relevant database items: $P@K = T/K$, $R@K = T/G$. Reranking an unchanged top-K set cannot alter P@K/R@K, but can improve P at smaller ranks or AP. A relevant item not retrieved contributes no achieved recall.

Raw ranking AP: relevance $r_k \in \{0,1\}$, cumulative $t_k = \sum_{j\le k} r_j$; $P_k = t_k/k$, $R_k = t_k/G$:

$$AP_{raw} = \frac{1}{G}\sum_k r_k P_k.$$

Use specified total G; not automatically number of returned relevant results. Unreturned positives contribute 0. Example [TP,FP,TP], G=3 gives $(1+2/3)/3 = 5/9$, achieved recall 2/3. If all G retrieved, raw AP = average precision at relevant ranks. Sorting/ties must follow evaluation policy.

All-points interpolated AP: precision envelope $P_{env}(R) = \max_{\tilde{R}\ge R} P(\tilde{R})$, then sum positive recall jumps $\sum_i (R_i - R_{i-1}) P_{env}(R_i)$; unreached recall contributes 0. Right-to-left cumulative max yields envelope. Can differ from raw AP. Older 11-point AP averages envelope at recalls 0,0.1,. . . ,1; COCO uses its own sampled-recall implementation. Never replace a stated rule with another.

mAP axis: retrieval = mean query AP, $Q^{-1}\sum_q AP_q$; detection = mean class AP under common matching/threshold conventions. mAP@0.5 uses IoU threshold .5. COCO-style average also spans IoU .50:.05:.95 (10 thresholds), with specific recall sampling. Mean of raw box precisions is not mAP.

```
tp = np.asarray(tp, dtype=float) # score order
t = tp.cumsum()
p = t / np.arange(1, len(tp)+1)
r = t / G
ap_raw = np.sum(p*tp) / G
env = np.maximum.accumulate(p[::-1])[::-1]
ap_interp = np.sum(np.diff(np.r_[0,r])*env)
```

Here each TP increments recall by 1/G; G>0 and specified missing-positive convention required. Classes/queries with G=0 need the evaluator's separate rule.

7. Boxes, IoU and detection evaluation

Continuous corner box $[x_1, y_1, x_2, y_2]$, width $x_2 - x_1$, height $y_2 - y_1$, nonnegative valid dimensions. Centre box (c_x, c_y, w, h) has corners $(c_x \pm w/2, c_y \pm h/2)$. Do not add 1 unless inclusive integer-pixel convention stated.

$$w_I = \max(0, \min(x_{2A}, x_{2B}) - \max(x_{1A}, x_{1B})),$$
$$h_I = \max(0, \min(y_{2A}, y_{2B}) - \max(y_{1A}, y_{1B})),$$
$$IoU = \frac{w_I h_I}{A_A + A_B - w_I h_I}.$$

Jaccard index; 0 disjoint, 1 identical positive-area regions; IoU can apply to masks. IoU unaffected by common translation/uniform scale of both regions, unlike raw unnormalized coordinate squared error. Degenerate zero-union needs stated rule.

AP matching: per class, sort detections descending score; match qualifying overlap to an *unmatched* GT under stated assignment rule; one GT can be claimed once. TP correct class plus sufficient IoU; wrong class/low IoU/duplicate = FP. Unmatched GT = FN. Duplicate overlapping an already claimed object does not become another TP unless validly assigned to another GT. NMS is prediction postprocessing, not GT evaluation matching.

Thresholds differ: confidence cutoff selects candidates; NMS IoU threshold removes redundant predictions; evaluation IoU threshold decides correctness. These are not the same setting. Early FP lowers some precision but does not force AP=0. For fixed predictions a tighter GT IoU criterion cannot create a previously invalid match, but score/assignment details still matter.

Segmentation scores: pixel accuracy dominated by large classes/background; per-class IoU $TP/(TP+FP+FN)$, mean IoU = average class IoU. Dice/F1 $2TP/(2TP+FP+FN) = 2IoU/(1+IoU)$ for same binary counts. Instance evaluation also needs mask-level one-to-one matching.

Metric example: TP=6, FP=2, FN=4, TN=8 (N=20): P=.75, R=.60, F1=2/3, accuracy=.70. Positive-region IoU=6/12=.50, Dice=12/18=2/3. Adding 80 TN raises accuracy to .94 but leaves P,R,F1,IoU unchanged–background abundance can hide poor positive performance.

AP variants differ: [FP,TP,TP], G=2: raw AP=$(1/2+2/3)/2 = 7/12$; all-points envelope AP=2/3. Raising precision at later recall can increase earlier envelope values, not raw rank precision.

8. Activations, losses and backpropagation

Neuron: affine preactivation $z = w^\top x + b$, nonlinear output $a = \phi(z)$. MLP = Multi-Layer Perceptron (FC layers + nonlinearities). Without nonlinearities, composed affine layers remain one affine map; depth alone does not add nonlinear expressive power.

Activations: ReLU $\max(0, z)$, derivative 1 if z>0, 0 if z<0 (often 0 at z=0). Leaky ReLU $\max(z, \alpha z)$, $0 < \alpha < 1$, negative derivative α. Sigmoid $\sigma = 1/(1+e^{-z})$, derivative $\sigma(1-\sigma)$; tanh derivative $1 - \tanh^2 z$. Sigmoid/tanh saturate at large $|z|$, weakening gradients.

Softmax: $p_c = e^{z_c}/\sum_j e^{z_j}$; subtract max z for stable exponentials. Nonnegative sum-1 outputs are not guaranteed calibrated probabilities. Softmax is shift-invariant to common logit offset, not common scaling; argmax logits = argmax probabilities.

Losses: multiclass CE $\ell = -\log p_y = -z_y + \log\sum_j e^{z_j}$; gradient logits $p - \text{onehot}(y)$. Binary CE $-y\log p - (1-y)\log(1-p)$; sigmoid-logit derivative p-y. Linear regression identity output, MSE = $N^{-1}\sum(\hat{y} - y)^2$, derivative $2(\hat{y} - y)/N$. Sum vs mean conventions change gradient scale. PyTorch `CrossEntropyLoss`: logits + integer labels; `NLLLoss`: *log probabilities* + integer labels. Don't apply softmax twice.

Chain rule: $\partial\ell/\partial x = (\partial z/\partial x)^\top(\partial\ell/\partial z)$. Affine batch row notation $Z = XW^\top + b$: $\nabla_W \ell = \delta^\top X$, $\nabla_X \ell = \delta W$, $\nabla_b \ell = \sum_{batch} \delta$. Multiply by activation derivative at each layer. Shared variables sum all path contributions (CNN details page 6).

9. Optimization, rate and initialization (Lab5)

GD/SGD: $\theta \leftarrow \theta - \eta\nabla\ell$; full batch uses all data; stochastic/mini-batch estimates gradient with noise. Momentum convention $v_t = \beta v_{t-1} + g_t$, $\theta_t = \theta_{t-1} - \eta v_t$ (some include $1-\beta$; follow definition). Epoch = one pass through train; step = one update. Shuffling reduces order effects; batch size influences noise/work, not trainable weight count.

Adam = Adaptive Moment Estimation (Kingma/Ba):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\hat{m}_t = m_t/(1-\beta_1^t), \quad \hat{v}_t = v_t/(1-\beta_2^t),$$
$$\theta_t = \theta_{t-1} - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Elementwise moments, common base learning rate still matters. Large η: overshoot/oscillation/divergence; tiny: slow progress. Optimizer ranking is task-dependent; Adam is not universally better than SGD. Compare validation and equal/declared training budgets.

Initialization: randomize interchangeable hidden weight vectors to break symmetry; identical/zero vectors can learn identically. Zero biases with random weights are fine. All-zero ReLU units can block useful branch gradients, but some output biases can still update. Xavier/Glorot variance $\approx 2/(fan_{in} + fan_{out})$; uniform bound $\sqrt{6/(fan_{in} + fan_{out})}$. Kaiming/He ReLU variance $\approx 2/fan_{in}$ (normal std its square root); convolution fan-in = $K_h K_w C_{in}$, not image area.

Gradient problems: products of small derivatives → vanishing; large → exploding/unstable. Help: suitable initialization/normalization, non-saturating activations, residual paths; gradient clipping limits large gradients, not restore vanished ones. Dying ReLU = preactivation stays nonpositive; appropriate rate/init, leaky ReLU/normalization can help; residuals do not guarantee unit revival.

10. Generalization, modes and diagnostic code

Underfit: poor train and validation (capacity or optimization/constraints). Overfit: strong train, worse held-out, increasing gap. Deeper model with higher *train* error can be optimization difficulty (degradation), not overfit. More depth is not automatically better. Mitigate overfit by augmentation, weight regularization, dropout, early stopping, more representative data/reduced capacity; validate, no guarantees.

Augmentation: label-preserving transforms or correctly updated boxes/masks; not arbitrary rotations/colour changes for every task. Dropout randomly drops activations during train; ordinary eval disables it with defined scaling. BatchNorm has different train/eval statistics. `model.eval()` does not disable autograd; `no_grad()` does not set eval mode or freeze weights. Freezing = no parameter updates (`requires_grad=False`/optimizer selection).

```
model.train()
for x,y in train_loader:
    optimizer.zero_grad()  # gradients, not weights
    logits = model(x)
    loss = criterion(logits,y)
    loss.backward()        # accumulates derivatives
    optimizer.step()       # updates parameters
model.eval()
with torch.no_grad():
    logits = model(x)
    pred = logits.argmax(dim=1)
```

Total accuracy = total correct/total examples, not unweighted mean batch accuracy with unequal final batch. Per-channel input standardization $(x-\mu)/\sigma$ differs from min-max $(x - \min)/(\max - \min)$; use pretrained statistics and valid nonzero denominators. Grad-CAM inspects class-related spatial influence, not complete causal explanation (page 6).

Stable softmax/CE from logits, one sample and class index y:

```
a = logits - np.max(logits)
prob = np.exp(a) / np.exp(a).sum()
ce = -a[y] + np.log(np.exp(a).sum())
dz = prob.copy()
dz[y] -= 1              # gradient of CE wrt logits
```

For logits (0,0), p=(.5,.5), CE=log 2, gradient=(-.5,.5) if y=0. Confident correct logits reduce CE; confident incorrect increase it. Dataset mean loss should be sample-weighted too. Batches n=4 accuracy1 and n=1 accuracy0 give unweighted .5, actual4/5=.8.

L2 penalty: adding $\lambda\|\theta\|^2/2$ adds gradient $\lambda\theta$; plain SGD update $\theta \leftarrow (1-\eta\lambda)\theta - \eta g$. Adaptive-optimizer decoupled weight decay is not automatically identical to this penalty. Frozen weights can coexist with changing BN running statistics in train mode; decide mode as well as parameter updates.

1. CNN mechanics: shapes, parameters and work

CNN = Convolutional Neural Network: local receptive fields, shared kernels at spatial positions, many output-channel feature maps. Standard "convolution" forward is cross-correlation:

$$Y_{o,y,x} = b_o + \sum_{c,a,b} W_{o,c,a,b} X_{c,sy+a-p,sx+b-p}.$$

Each output filter spans all C_{in} channels; sum channels before activation. Weights shape $(C_{out}, C_{in}, K_h, K_w)$ in PyTorch; activations (N, C, H, W), common NumPy HWC differs. Channel count = number of filters, not spatial kernel size/depth in z-space.

Output size per axis: kernel K, stride S, left/right padding P_l, P_r, dilation D, effective kernel $K_{eff} = D(K-1)+1$:

$$N_{out} = \left\lfloor \frac{N + P_l + P_r - K_{eff}}{S} \right\rfloor + 1.$$

No dilation, symmetric P: $\lfloor (N+2P-K)/S \rfloor + 1$. At stride 1 odd K, P=$(K-1)/2$ preserves size. "Same" at larger stride commonly targets $\lceil N/S \rceil$ and may need asymmetric padding; inspect definition. Stride changes outputs/compute, *not* shared parameters. Activation volume = batch×$H_o W_o C_o$, different from parameter count.

Counts including bias: dense conv $C_o(K_h K_w C_i + 1)$; FC $D_o(D_i+1)$; 1×1 $C_o(C_i+1)$; biases one/output channel or FC unit, not one/output pixel. Pool/ReLU/flatten/add/concat have 0 ordinary learned parameters. BatchNorm affine has $2C$ (gamma/-beta); running means/variances are state, not trained parameters. Omit +bias only if absent. The supplied textbook caption incorrectly repeats C_i; correct kernel product includes C_o.

Compute: dense conv MACs $H_o W_o C_o K_h K_w C_i$ per image, excluding biases/nonlinearities. Multiply+add counted separately ≈ 2 FLOPs/MAC. Use *output* positions, not input unless size-preserving. Larger stride reduces MACs but same weights; batch multiplies work. Compare like counts (weights vs params vs activations vs MACs).

FC equivalence: whole $H \times W \times C$ input to D outputs = conv kernel H×W, $C_{out} = D$, no padding, one output position. Shared FC on nonoverlapping K×K patches = K×K conv stride K. Reusing one FC across patches does not multiply weight count by number of patches. A locally connected unshared layer does multiply distinct per-position weights.

BatchNorm: per-channel normalize using training batch/spatial statistics, learned γ, β:

$$y = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta.$$

Ordinary eval uses stored statistics, not current test batch. Instance/layer/group norm use different within-sample entry groups; check tensor axes. BN has $2C$ trainable affine params; running stats are buffers. Normalization can improve training stability, not guarantee no exploding/vanishing values.

1×1/bottleneck: mixes channels at each pixel, no direct spatial mixing; not identity/parameter-free. Reduce channels before expensive 3×3 then expand. Two stride-1 3×3 layers cover 5×5, three cover 7×7; at constant C no bias, $18C^2$ vs $25C^2$, $27C^2$ vs $49C^2$, plus intermediate nonlinearities; changing channels can alter savings.

3D conv: kernel (K_t, K_h, K_w), weights $(C_o, C_i, K_t, K_h, K_w)$; params $C_o(K_t K_h K_w C_i + 1)$. Frames T are a temporal axis, not $C_i = T$ unless explicitly packed as channels. Output shape per axis same formula; MACs $T_o H_o W_o C_o K_t K_h K_w C_i$. Framewise 2D alone does not mix time. (2+1)D spatial then temporal weights $K_h K_w C_i C_m + K_t C_m C_o$, plus biases; not always fewer without specifying C_m.

2. Pooling, receptive fields and equivariance

Max/average pooling operates per channel; no shared learned kernel. Max nonlinear, average fixed linear; both can downsample. Global average pool outputs one mean/channel, no params. Pooling gives limited local robustness, not automatic exact translation invariance. Stride/borders break arbitrary shift equivariance. Convolution shifts output with input (equivariance); invariant output would remain unchanged. Weight sharing improves cost/generalization but may not suit every fixed-position task.

Receptive-field recurrence: input $r = 1$, $j = 1$; each layer $r \leftarrow r + (K_{eff} - 1)j$, $j \leftarrow Sj$, in order. j = input-grid jump between outputs. Padding changes location/border coverage, not nominal support size. For L stride-1 K-wide layers $r = 1 + L(K-1)$. Pooling contributes its own K,S. Theoretical support ≠ measured effective influence.

Arithmetic example: 3×3 stride1 $r = 3$, $j = 1$; 2×2 pool stride2 $r = 4$, $j = 2$; 3×3 stride1 $r = 8$, $j = 2$. Don't just add K-1 after strides. A 32×32×3 input, 5×5 conv with 8 filters, P=0,S=1 gives 28×28×8, weights 600, params 608, MACs $28^2 \cdot 8 \cdot 75 = 470400$; S=3 gives 10×10×8, same 608 params.

3. Shared-kernel backpropagation

Let upstream $\delta = \partial \ell / \partial Y$; for correlation forward,

$$\frac{\partial \ell}{\partial W_{o,c,a,b}} = \sum_{n,y,x} \delta_{n,o,y,x} X_{n,c,sy+a-p,sx+b-p}.$$

Bias gradient sums δ across batch/positions. Input gradient accumulates δW from *every* window containing that input; overlapping patches add, not overwrite. ReLU gradient gates by forward preactivation sign. Max-pool sends gradient to chosen forward maxima (tie rule), average-pool distributes it evenly; repeated influence sums. Check stride/padding/kernel orientation before manually computing input gradient.

Tiny example: $X = \left[\begin{smallmatrix} 0 & -1 & 3 \\ 2 & .5 & 1 \end{smallmatrix}\right]$, $W = \left[\begin{smallmatrix} 0 & 1 \\ 1 & .5 \end{smallmatrix}\right]$, stride1, padding0, bias b=0 gives Y=(1.25,4). For $\delta = (1, 1)$:

$\nabla_X \ell = \left[\begin{smallmatrix} 0 & 1 & 1 \\ 1 & 1.5 & .5 \end{smallmatrix}\right]$, $\nabla_W \ell = \left[\begin{smallmatrix} -1 & 2 \\ 2.5 & 1.5 \end{smallmatrix}\right]$, $\nabla_b \ell = 2$.

Layer declarations (framework syntax to interpret, not required runtime):

```
conv = nn.Conv2d(3, 8, kernel_size=5, stride=1)
fc = nn.Linear(28*28*8, 10) # 62730 params
flat = features.flatten(start_dim=1) # retain batch
```

Flatten all nonbatch axes, not just channels; retain sample axis.

4. Detection: sliding windows and proposals

Dense classifier over position/scale/aspect ratio/orientation gives many overlapping candidates. Per axis window count $\lfloor (N+2P-K)/S \rfloor + 1$; total locations = rows×cols. **S4 Q4c box-evaluation formula:**

$N_{eval} = N_{locations}\, N_{aspect\ ratios}\, N_{scales}\, N_{classes}$.

9 locations×3 aspect ratios×4 scales×5 classes = **540 class–box evaluations**, but only108 distinct candidate boxes. Multiply by orientations too if varied; one multiclass window ≠ C distinct boxes. Runtime = candidates×time/candidate×images; seconds→hours divide3600. **Make sliding windows faster (S4 Q5c):** larger *stride* skips positions (roughly S^2 fewer in2D, exact floor counts above); *segmentation/region proposals* focus on likely object regions instead of all background windows, e.g. Selective Search. Lower search cost can reduce recall or localization precision; segmentation itself also costs computation. Shared whole-image CNN features avoid repeatedly computing overlapping crop features.

Selective Search: class-agnostic external heuristic proposals: oversegment, hierarchically merge neighbouring regions using colour/texture/-size/fill similarities; use varied initial segmentations/features/scales to generate diverse boxes. Fewer promising candidates than exhaustive windows, not guaranteed complete recall or learned object classifier.

Greedy hard NMS: sort scores; keep highest remaining; suppress lower-score candidates whose IoU > τ (or stated ≥); repeat. Usually per class. Discarded boxes never suppress others later: A suppressing B and B overlapping C does not imply A suppresses C. Only IoU to *kept* boxes matters. Confidence threshold first/also is different. High crowd overlap can suppress genuine separate instances; NMS doesn't guarantee one box/true object.

5. R-CNN family, RoI pooling and RPN

R-CNN (Regions with CNN features): external Selective Search proposals → crop/warp each raw region → separate CNN computation → class SVM and box regression. Overlapping crops repeat expensive conv work; multistage training/caching. **Fast R-CNN**: one shared whole-image conv map → each external proposal's RoI pooling → FC class/background and box-regression heads; joint task loss, but external proposal bottleneck remains. **Faster R-CNN**: shared backbone + learned RPN proposals → second-stage RoI head. "Two-stage" does not mean every crop gets a separate raw-image backbone.

RoI pooling = Region of Interest pooling: map proposal to feature map; partition into fixed $H_r \times W_r$ bins, pool each/channel; fixed-size $H_r W_r C$ output from variable region. Quantization may affect alignment; Mask R-CNN adds a mask branch, with more accurate region alignment in common versions (no advanced implementation required).

RPN = Region Proposal Network: sliding learned predictor on shared map, A anchors/location with scales/aspects. Predict objectness and 4 box offsets per anchor; score-filter/NMS proposals, then second-stage classification/refinement. Objectness = object vs background, not final C-way class. With two-logit objectness: 2A cls +4A reg channels; with scalar sigmoid: A+4A. At $H_f W_f$ positions, $H_f W_f A$ anchors; formula depends on stated representation.

Anchor-relative regression (common convention):

$t_x = (x - x_a)/w_a$, $t_y = (y - y_a)/h_a$,

$t_w = \log(w/w_a)$, $t_h = \log(h/h_a)$.

Decode $x = x_a + w_a t_x$, $y = y_a + h_a t_y$, $w = w_a e^{t_w}$, $h = h_a e^{t_h}$. Anchor reference shape differs from predicted box; anchors don't increase trainable backbone parameters unless head channels/weights change. Match labels/positive thresholds are detector-specific, not universal IoU settings.

Combined head loss: classification + weighted box regression for designated positives (e.g. smooth L1). smoothL1$(r) = r^2/2$ if $|r| < 1$, else $|r| - 1/2$ in one convention. Ignore/background examples may contribute classification without positive regression.

6. One-stage detectors, SSD and YOLO versions

One-stage: directly predict dense class/box outputs from image features, no proposal-then-RoI classification stage. Both one-/two-stage can share a backbone; speed/accuracy depends on design/settings, not an absolute law. **SSD = Single Shot MultiBox Detector**: default boxes at multiple feature-map resolutions/scales/aspect ratios for different object sizes.

Original YOLO v1 = You Only Look Once: S×S grid, B boxes/cell, one shared C-class vector/cell:

output $= S \times S \times (5B + C)$, $\#boxes = S^2 B$.

Object assigned to cell containing its centre; responsible predictor chosen by overlap in training. Each box: x,y,w,h,confidence; confidence = Pr$(object)\,IoU$; class score = Pr$(class \mid object)$ Pr$(object)\,IoU$. Confidence is *predicted*; no GT is provided at inference. Use v1 cell/image-normalized coordinates, not later anchor equations. Example S=7,B=2,C=20 ⇒ 7×7×30 = 1470 scalars, 98 boxes. Close same-cell objects challenge v1.

Do not generalize v1 equations: later per-anchor variants may have $B(5+C)$ channels/cell, different objectness/IoU targets and decoding. Read version and channel convention before counting; both box count and scalar count are different quantities. Filter class scores then NMS under stated policy.

Backbone/neck/head: backbone extracts features (e.g. VGG, ResNet); neck combines features/scales (FPN, etc.); head outputs classes/boxes/objectness. FPN = Feature Pyramid Network: top-down upsampled coarse semantics + lateral fine-spatial features, multiscale predictions; not just an image pyramid.

YOLOv4 reading: bag of freebies = training-only improvements with no inference cost (augmentation, loss/training changes); bag of specials = small-cost architectural/postprocessing improvements. Coordinate MSE is scale-sensitive; IoU loss uses relative overlap. Plain $1 - IoU$ can have poor/zero gradient for disjoint boxes; GIoU adds enclosure, DIoU centre distance, CIoU also aspect consistency. Detailed newer detector implementation not required.

NMS example with actual geometry: A=(0,0,4,2), B=(1,0,5,2), C=(2,0,6,2), scores3,2,1. IoU(A,B)=IoU(B,C)=.6; IoU(A,C)=1/3. At suppression threshold .5, keep A, discard B, keep C. Discarded B cannot suppress C. If treating all overlaps transitively, answer would be wrong.

RPN/head counts: 20×20 feature map, 9 anchors/location gives3600 candidates. With two objectness logits and four offsets/anchor, heads have18 and36 channels. 1×1 heads on256 input channels use $(256+1)(18+36) = 13878$ params including biases, separate from shared backbone/sliding conv. Five foreground classes typically need6 class/background logits in a two-stage head; class-specific vs class-agnostic box-regression widths depend on stated design.

7. Architectural patterns

LeNet: early backprop-trained conv digit/character recognition, Yann LeCun. **AlexNet**: 2012 ImageNet breakthrough, Alex Krizhevsky/Ilya Sutskever/Geoffrey Hinton; GPU training, ReLU/deeper CNN. **VGG**: Simonyan/Zisserman (Visual Geometry Group), stacks of 3×3 convs + pooling, often increasing channels; VGG16/19 counts learned layers in named architectures. Small stacked kernels enlarge receptive field with extra nonlinearities and often fewer weights than single large kernels for suitable channels.

8. ResNet: addition, shortcuts and bottlenecks

Residual block: $y = x + F(x)$ before optional final activation. Learns residual, can make F=0 for identity *preactivation*. If final ReLU, output is ReLU(x), not identity on arbitrary negative x. Direct addition requires matching spatial/channel shape; use projection/downsampling $y = P(x) + F(x)$ when changing resolution/channels.

Why: deep plain networks can have higher train error from optimization difficulty; residual direct paths ease identity-like mapping and gradient propagation, not automatically eliminate overfit or every gradient problem. $\partial y/\partial x = I + \partial F/\partial x$ before activation; skip has no learned parameters if literal identity, projection does.

Basic block: commonly two 3×3 convs; **bottleneck**: 1×1 reduce $C_i \to C_b$, 3×3 $C_b \to C_b$, 1×1 expand $C_b \to C_o$. Weight count $C_i C_b + 9C_b^2 + C_b C_o$, plus present biases/BN/projection. Shape-changing 1×1 shortcut weights $C_i C_o$ (stride can downsample). Sum branches does not concatenate/increase channels. ResNet authors: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun.

9. Encoder–decoder, segmentation and autoencoders

Encoder: compress/downsample spatial representations, grow context. **Decoder**: upsample/reconstruct dense output; low spatial resolution alone loses fine boundaries. **FCN = Fully Convolutional Network**: spatial output from convolutional layers rather than one fixed-size classifier head; supports dense prediction within architecture constraints.

Upsampling: nearest/bilinear fixed interpolation has no trainable parameters. Transposed convolution is learned transpose of convolution's linear matrix (before bias), not mathematical inverse. Output per axis (common convention): $(N-1)S - 2P + D(K-1) + P_{out} + 1$. "Output padding" chooses output shape, not adding learned samples. Uneven overlap can cause checkerboard artifacts; resizing+conv is an alternative.

SegNet: store encoder max-pool indices, unpool into those positions, then refine. Unpooling does not restore lost nonmax information. **U-Net**: corresponding encoder high-resolution features concatenate with upsampled decoder features at matched spatial scales. Channels $C_1 + C_2$ (not C); ResNet adds same-shape tensors and retains C. Crop/resize/shape alignment can be necessary before concat.

Semantic output: C logits/pixel, per-pixel softmax/CE (or binary sigmoid/BCE); ignore/weight labels as specified. Box classifier is not a segmentation mask. **Mask R-CNN**: instance mask branch alongside class/box; separate masks distinguish overlapping same-class objects. **Panoptic**: unify thing instances and stuff semantic regions.

Autoencoder: encoder $x \to z$, decoder $z \to \hat{x}$, train to reconstruct input (MSE/BCE etc.); no separate semantic labels required. Latent bottleneck restricts representation, not guarantee semantic or perfect reconstruction; over-capacity can simply learn copying. Self-supervised rendering/consistency uses observations as targets instead of manual labels for every output; assumptions/visibility still matter.

10. Transfer learning, Grad-CAM and course names

Transfer: reuse pretrained features/weights (often ImageNet 1000 classes); replace head for new class meanings/count. Freeze backbone/train new head for limited similar-domain data, fine-tune selected/all layers for more data/domain shift, usually choose suitable lower rate. Validate; no guarantee frozen/random/fine-tuned ordering. Match expected RGB/BGR, scale, mean/std and size; freezing and eval/no-grad are distinct (page 5).

Grad-CAM: selected conv activations A^k, class score s_c, pooled gradients $\alpha_k^c = (1/HW)\sum_{i,j} \partial s_c / \partial A_{ij}^k$; heatmap ReLU$(\sum_k \alpha_k^c A^k)$, upsample for display. Helps inspect shortcuts/error attention; coarse localization, not guaranteed causal explanation/precise object mask. Do not disable gradients before computing class-gradient visualization.

Name/term reminders: photometric stereo–Robert Woodham; Lambert–cosine diffuse law; Phong–ambient/diffuse/specular; Canny–John Canny (1986); Harris/Stephens–corner (1988); Shi/Tomasi–minimum eigenvalue; Lucas/Kanade–local flow (1981); Horn/Schunck–global smooth flow; Lowe–SIFT/ratio; Fischler/Bolles–RANSAC (1981); McCulloch/Pitts–1943 artificial neuron; Rosenblatt–perceptron; Girshick–R-/Fast R-CNN; Ren et al.–Faster R-CNN; Redmon et al.–original YOLO (2016); Glorot/Bengio–Xavier; He–Kaiming/ResNet; Kingma/Ba–Adam.

Exam-use checks: read convention, units, shapes, padding, threshold strictness, AP variant and detector version first; compute float/full precision, round final value only; exact counts stay integers. This is a course-aligned reference, not an exhaustive guarantee/prediction. Guest Lecture 13 and optional-only advanced implementations are excluded. All 12 main decks, Seminars 1–5 and Labs 0–5 inform these six sides.

Transfer/head arithmetic: 512 input features to 7 new classes needs $7(512+1) = 3591$ head params with bias; old 1000-class head had513000. Replacing head doesn't change backbone features' channel count. A fully frozen backbone + this head has3591 trainable params (unless other trainable modules are retained), although total stored model params are much larger.

Skip shapes: 64-channel encoder +128-channel decoder concatenation gives192 channels; direct addition is invalid without shape adaptation. A subsequent 1×1 conv to32 channels has $32(192+1) = 6176$ params with bias, not192+32. Upsampling changes spatial resolution, not automatically number of channels/-classes.